

RAY TRACING

in One Weekend

‘Ray Tracing in One Weekend’를 C++로 따라 구현하며 정리한 스터디 발표입니다.
PPM 이미지 출력부터 재질, 카메라, 최종 렌더까지 13개 챕터를 다룹니다.

Contents

목차

02 Output an Image
 이미지 출력

03 The vec3 Class
 vec3 클래스

04 Rays, a Simple Camera and Background
 광선, 간단한 카메라, 그리고 배경

05 Adding a Sphere
 구 추가하기

06 Surface Normals and Multiple Objects
 표면 법선과 다중 객체

07 Moving Camera Code Into Its Own Class
 카메라 코드를 자체 클래스로 이동

08 Antialiasing
 안티 앨리어싱

09 Diffuse Materials
 확산 재질

10 Metal
 금속

11 Dielectrics
 유전체

12 Positionable Camera
 위치 조정 가능한 카메라

13 Defocus Blur
 초점 흐림

14 Final Render
 최종 렌더

* 챕터 번호는 원서 기준 (1장 Overview 제외)

Output an Image

이미지 출력

02

1. PPM 이미지 포맷

PPM은 이미지 포맷 중 하나다. 픽셀은 왼쪽에서 오른쪽으로, 위에서 아래 순서로 작성된다.

2. 256 × 256 예시 이미지

픽셀 색상은 0~255 사이의 정수로 표현할 수 있으므로 예시 이미지의 크기를 width 256, height 256으로 설정한다.

3. 0~1 실수로 계산, 0~255 정수로 출력

관례적으로 r, g, b는 0과 1 사이의 실수로 표현한다. 0~255 정수를 실수로 바꿔 계산하고, 출력할 때 다시 0~255 정수로 변환한다.

4. 빌드 결과를 이미지로

실행 결과를 리다이렉션(>)으로 .ppm 파일에 저장한다. 파일 이름은 상관없다.

```
D:\C++\RayTracingStudy\Phase1\x64\Debug>Phase1.exe > Phase1.ppm
Done.
```

```
D:\C++\RayTracingStudy\Phase1\x64\Debug>|
```

```
int main()
{
    // Image;
    int ImageWidth = 256;
    int ImageHeight = 256;

    // Render
    std::cout << "P3\n" << ImageWidth << ' ' << ImageHeight << "\n255\n";

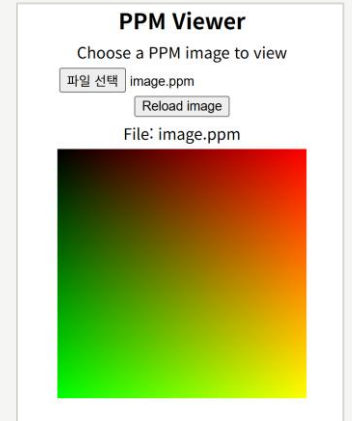
    for (int j = 0; j < ImageHeight; j++)
    {
        for (int i = 0; i < ImageWidth; i++)
        {
            // r,g,b 값은 관례적으로 0과 1사이의 실수값으로 표현합니다.
            auto r = double(i) / (ImageWidth - 1);
            auto g = double(j) / (ImageHeight - 1);
            auto b = 0.0;

            // 출력을 할 때 0~255 사이의 정수값으로 스케일링 합니다.
            int ir = int(255.999 * r);
            int ig = int(255.999 * g);
            int ib = int(255.999 * b);

            std::cout << ir << ' ' << ig << ' ' << ib << '\n';
        }
    }

    std::clog << "\rDone.          \n";

    return 0;
}
```



PPM Viewer로 확인한 출력

PPM Viewer
cs.rhodes.edu/welshc/COMP141_F16/ppmReader.html

The vec3 Class

vec3 클래스 — 구조와 멤버 연산자

Vector3 구조체

길이가 3인 double 배열을 멤버로 가진다. 배열의 0, 1, 2번째는 x, y, z 좌표를 의미한다. 기본 생성 시 원점으로 초기화되고, x, y, z 좌표를 지정해서 생성할 수도 있다. 편리하게 사용할 수 있도록 연산자 오버로딩이 되어 있다.

```
struct Vector3
{
    Vector3() : E{ 0,0,0 } {}
    Vector3(double e0, double e1, double e2) : E{ e0,e1,e2 } {}
```

```
    double E[3];
};
```

using Point3 = Vector3;

벡터와 점을 구분하기 위해 정의한다.

멤버 함수 · 연산자

<code>operator-()</code>	벡터의 각 요소에 -1을 곱한다.
<code>operator[](int i)</code>	인자로 들어온 인덱스의 요소를 반환한다.
<code>operator+=(v)</code>	벡터끼리 += 연산을 할 수 있게 해준다.
<code>operator*=(t)</code>	실수 t와 *= 연산을 할 수 있게 해준다.
<code>operator/=(t)</code>	실수 t와 /= 연산. 위에서 정의한 *= 연산자에 t의 역수를 넘겨 호출한다.
<code>Length()</code>	벡터의 길이를 반환한다.
<code>LengthSquared()</code>	벡터 길이의 제곱 $x^2 + y^2 + z^2$ 을 반환한다.

`v : const Vector3& · t : double`

The vec3 Class

vec3 클래스 — 유틸리티 함수

03

비멤버 유틸리티 함수

$u, v : \text{const Vector3\&}$ · $t : \text{double}$

<code>operator<<(out, v)</code>	cout 등으로 벡터를 출력할 수 있게 해준다.
<code>operator+(u, v)</code>	두 벡터의 덧셈
<code>operator-(u, v)</code>	두 벡터의 뺄셈
<code>operator*(u, v)</code>	두 벡터의 (요소별) 곱셈
<code>operator*(t, v)</code>	실수 t와 벡터의 곱셈
<code>operator*(v, t)</code>	순서를 바꾼 경우도 가능하게 했다.
<code>operator/(v, t)</code>	벡터 v를 실수 t로 나눈다. 내부에선 t의 역수를 취하고 <code>operator*</code> 를 호출한다.
<code>Dot(u, v)</code>	두 벡터의 내적. 결과는 double 실수
<code>Cross(u, v)</code>	두 벡터의 외적. 결과는 벡터
<code>UnitVector(v)</code>	단위 벡터

```
// 벡터 유틸리티 함수들
inline std::ostream& operator<<(std::ostream& out, const Vector3& v)
{
    return out << v.E[0] << ' ' << v.E[1] << ' ' << v.E[2];
}

inline Vector3 operator+(const Vector3& u, const Vector3& v)
{
    return Vector3(u.E[0] + v.E[0], u.E[1] + v.E[1], u.E[2] + v.E[2]);
}

inline Vector3 operator-(const Vector3& u, const Vector3& v)
{
    return Vector3(u.E[0] - v.E[0], u.E[1] - v.E[1], u.E[2] - v.E[2]);
}

inline Vector3 operator*(const Vector3& u, const Vector3& v)
{
    return Vector3(u.E[0] * v.E[0], u.E[1] * v.E[1], u.E[2] * v.E[2]);
}

inline Vector3 operator*(double t, const Vector3& v)
{
    return Vector3(t * v.E[0], t * v.E[1], t * v.E[2]);
}
```

```
inline Vector3 operator*(const Vector3& v, double t)
{
    return t * v;
}

inline Vector3 operator/(const Vector3& v, double t)
{
    return (1 / t) * v;
}

inline double Dot(const Vector3& u, const Vector3& v)
{
    return u.E[0] * v.E[0] + u.E[1] * v.E[1] + u.E[2] * v.E[2];
}

inline Vector3 Cross(const Vector3& u, const Vector3& v)
{
    return Vector3(
        u.E[1] * v.E[2] - u.E[2] * v.E[1],
        u.E[2] * v.E[0] - u.E[0] * v.E[2],
        u.E[0] * v.E[1] - u.E[1] * v.E[0]);
}

inline Vector3 UnitVector(const Vector3& v)
{
    return v / v.Length();
}
```

벡터의 사칙연산 및 내적 · 외적 계산 설명은 생략

Rays, a Simple Camera and Background

04

광선, 간단한 카메라, 그리고 배경 — Ray 클래스와 이미지 설정

1 Ray 클래스

```
class Ray
{
public:
    Ray() {}

    Ray(const Point3& origin, const Vector3& direction)
        : mOrig(origin), mDir(direction) {}

    const Point3& Origin() const { return mOrig; }
    const Vector3& Direction() const { return mDir; }

    Point3 At(double t) const
    {
        return mOrig + t * mDir;
    }

private:
    Point3 mOrig;
    Vector3 mDir;
};
```

mOrig 광선의 원점

mDir 광선의 방향

At 함수가 광선을 표현한다고 하면, 매개변수 t의 값에 따라 광선 위를 움직인다고 생각하면 된다.

$$P(t) = mOrig + t \cdot mDir$$

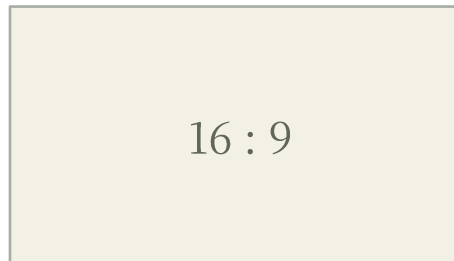
2 이미지 설정

```
int main()
{
    // Image
    auto aspectRatio = 16.0 / 9.0;
    int imageWidth = 400;

    // 이미지 높이를 계산하고 최소 1이 되도록 합니다.
    int imageHeight = int(imageWidth / aspectRatio);
    imageHeight = (imageHeight < 1) ? 1 : imageHeight;
```

렌더링에 사용될 이미지를 정의한다. 일반적인 비율인 16:9를 사용한다. 값을 바꿔도 비율을 일정하게 유지할 수 있도록 너비를 먼저 정의한 후 높이를 계산한다. 이때 높이의 최소값이 1이 되도록 한다.

imageWidth = 400



imageHeight = 400 / (16/9) = 225

Rays, a Simple Camera and Background

광선, 간단한 카메라, 그리고 배경 — 카메라와 뷰포트

04

1. 뷰포트

카메라와 뷰포트를 설정한다. 뷰포트는 픽셀을 포함하며 광선이 통과하는 가상의 직사각형이다.

2. 같은 종횡비

픽셀이 가로와 세로로 동일한 간격을 가지면 뷰포트와 렌더링된 이미지는 같은 종횡비를 가진다.

3. 높이 먼저, 너비는 계산

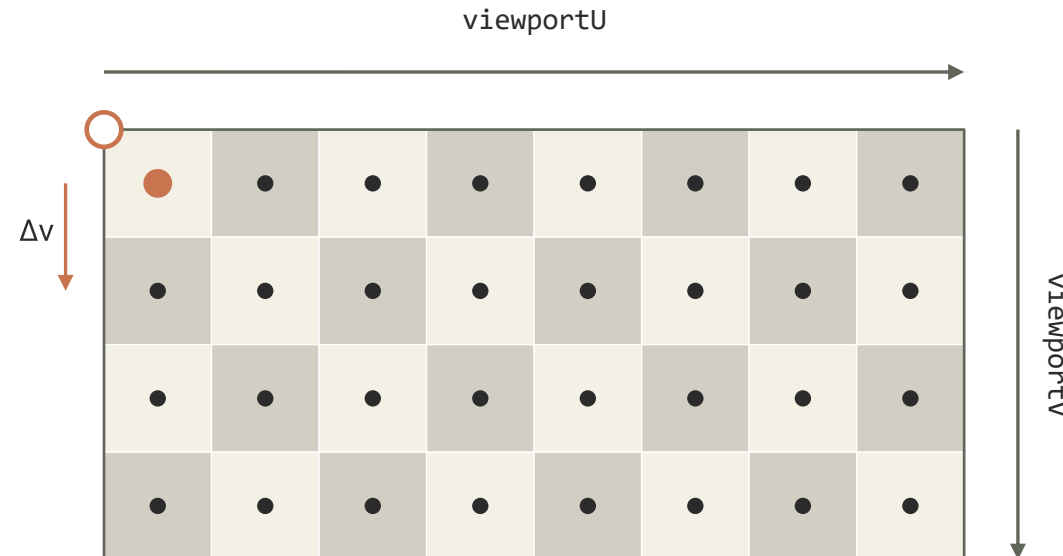
이미지와 동일하게 뷰포트도 먼저 높이를 설정하고 너비를 계산한다.

```
// Camera
auto focalLength = 1.0;
auto viewportHeight = 2.0;
auto viewportWidth = viewportHeight * (double(imageWidth) / imageHeight);
auto cameraCenter = Point3(0, 0, 0);

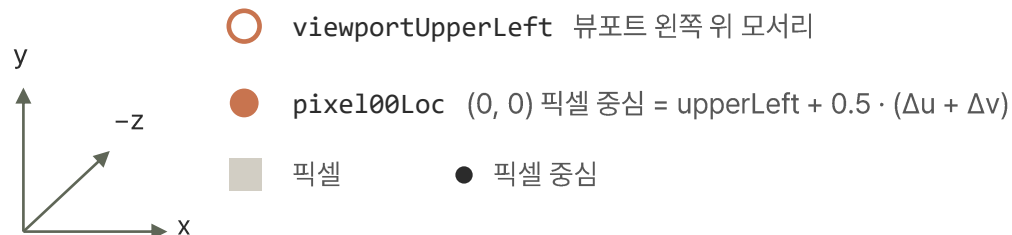
// 뷰포트의 수평 및 수직 가장자리를 가로지르는 벡터를 계산합니다.
auto viewportU = Vec3(viewportWidth, 0, 0);
auto viewportV = Vec3(0, -viewportHeight, 0);

// 픽셀 간 수평 및 수직 델타 벡터를 계산합니다.
auto pixelDeltaU = viewportU / imageWidth;
auto pixelDeltaV = viewportV / imageHeight;

// 왼쪽 위 픽셀의 위치를 계산합니다.
auto viewportUpperLeft = cameraCenter - Vec3(0, 0, focalLength) - viewportU / 2 - viewportV / 2;
auto pixel00Loc = viewportUpperLeft + 0.5 * (pixelDeltaU + pixelDeltaV);
```



카메라는 오른손 좌표계, Δu 방향으로 바라본다



Rays, a Simple Camera and Background

광선, 간단한 카메라, 그리고 배경 — 배경 그리기

04

1. 광선 만들기

픽셀 좌표와 카메라의 중심을 알고 있으므로 눈(시점)에서 픽셀까지의 광선을 구할 수 있다.

2. RayColor

광선의 Y 성분을 색상으로 표현한다.

3. 0 ~ 1 범위로

방향을 단위 벡터로 만든 뒤 $a = 0.5 \cdot (y + 1)$ 로 범위를 0에서 1 사이로 만들어준다.

4. 선형 보간

$(1 - a) \cdot \text{흰색} + a \cdot \text{파란색}$. 이미지의 위쪽으로 갈수록 색이 파래진다.

```
// Render
std::cout << "P3\n" << imageWidth << " " << imageHeight << "\n255\n";

for (int j = 0; j < imageHeight; j++)
{
    std::clog << "\rScanlines remaining: " << (imageHeight - j) << ' ' << std::flush;
    for (int i = 0; i < imageWidth; i++)
    {
        auto pixelCenter = pixel00Loc + (i * pixelDeltaU) + (j * pixelDeltaV);
        auto rayDirection = pixelCenter - cameraCenter;
        Ray r(cameraCenter, rayDirection);

        Color pixelColor = RayColor(r);
        WriteColor(std::cout, pixelColor);
    }
}
```

```
Color RayColor(const Ray& r)
{
    Vec3 unitDirection = UnitVector(r.Direction());
    auto a = 0.5 * (unitDirection.Y() + 1.0);

    return (1.0 - a) * Color(1.0, 1.0, 1.0) + a * Color(0.5, 0.7, 1.0);
}
```



결과 —
위로 갈수록 파래지는 배경

Adding a Sphere

구 추가하기

05

1. 구의 방정식

점 P에서 구의 중심 C까지의 벡터는 $C - P$. 내적의 정의를 사용하면 $(C - P) \cdot (C - P) = r^2$ 이 나온다.

2. t에 대한 이차방정식

광선 매개변수 t를 제외하고 모든 변수의 값이 정해져 있으므로, $P(t)$ 를 대입해 t를 중심으로 방정식을 정리한다. 판별식으로 교차 여부를 알 수 있다.

3. HitSphere

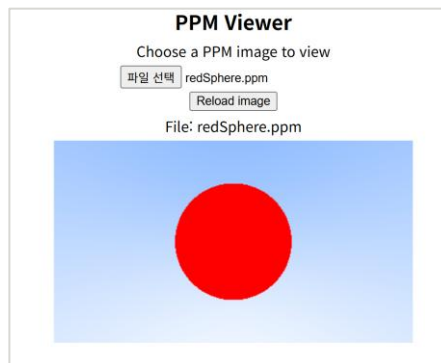
광선이 구와 만나면 참을 반환한다. RayColor에서는 이때 빨간색을 반환한다 — 빨간 원이 그려진다.

```
bool HitSphere(const Point3& center, double radius, const Ray& r)
{
    Vec3 oc = center - r.Origin();
    auto a = Dot(r.Direction(), r.Direction());
    auto b = -2.0 * Dot(r.Direction(), oc);
    auto c = Dot(oc, oc) - radius * radius;
    auto discriminant = b * b - 4 * a * c;
    return (discriminant >= 0);
}

Color RayColor(const Ray& r)
{
    if (HitSphere(Point3(0, 0, -1), 0.5, r))
        return Color(1, 0, 0);

    Vec3 unitDirection = UnitVector(r.Direction());
    auto a = 0.5 * (unitDirection.Y() + 1.0);

    return (1.0 - a) * Color(1.0, 1.0, 1.0) + a * Color(0.5, 0.7, 1.0);
}
```



빌드 결과 —
빨간 원이 그려진다

광선 $P(t) = \underbrace{Q}_{\text{점}} + \underbrace{t \cdot \text{방향}}_{\text{광선 계수}}$

$(C - P(t)) \cdot (C - P(t)) = r^2$ 을 만족하면 구에 닿는다:

$$(C - (Q + t \cdot d)) \cdot (C - (Q + t \cdot d)) = r^2$$

$$(-td + (C - Q)) \cdot (-td + (C - Q)) = r^2$$

$$(\vec{A} + \vec{B}) \cdot (\vec{A} + \vec{B})$$

$$\vec{A} \cdot (\vec{A} + \vec{B}) + \vec{B} \cdot (\vec{A} + \vec{B})$$

$$\vec{A} \cdot \vec{A} + 2\vec{A} \cdot \vec{B} + \vec{B} \cdot \vec{B}$$

$$(-td) \cdot (-td) + 2(-td) \cdot (C - Q) + (C - Q) \cdot (C - Q) = r^2$$

$$t^2 d \cdot d - 2td \cdot (C - Q) + (C - Q) \cdot (C - Q) = r^2$$

$$t^2 d \cdot d - 2td \cdot (C - Q) + (C - Q) \cdot (C - Q) - r^2 = 0$$

마지막 t에 대한 이차 방정식

$$\text{근의 공식} : \frac{b \pm \sqrt{b^2 - 4ac}}{-2a} \quad \text{판별식 } b^2 - 4ac$$

$$\rightarrow b^2 - 4ac = 0$$

$$\rightarrow b^2 - 4ac > 0$$

$$\rightarrow b^2 - 4ac < 0$$

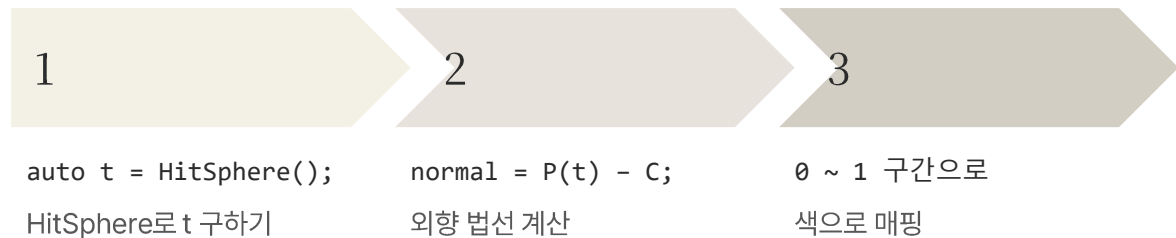
구 방정식 → t에 대한 이차방정식 유도

Surface Normals and Multiple Objects

06

표면 법선과 다중 객체 — 법선을 색으로

앞 장에서 구와 만나는 모든 광선을 빨간색으로 표현했다면, 이번에는 법선에 색을 매핑한다. 법선을 매핑하려면 충돌점이 필요하다.



1. 가장 가까운 충돌점

t가 음수일 때(판별식이 0보다 작음)는 고려하지 않고, 가장 가까운 충돌점을 사용한다. 근의 공식 $\pm$ 중 $-$ 를 쓰겠다는 뜻이다.

2. 식 단순화

근의 공식에서 b 대신 2h를 넣어서 식을 단순화할 수 있다. HitSphere 함수는 이후에 Sphere 클래스로 이동한다.

main.cpp — HitSphere 함수

```
if (discriminant < 0.0)
{
    return -1.0;
}

return ((h - std::sqrt(discriminant)) / a);
```

```
double HitSphere(const Point3& center, double radius, const Ray& ray)
{
    Vec3 originToCenter = center - ray.Origin();
    auto a = ray.Direction().LengthSquared();
    auto h = Dot(ray.Direction(), originToCenter);
    auto c = originToCenter.LengthSquared() - radius * radius;
    auto discriminant = h * h - a * c;

    if (discriminant < 0.0)
    {
        return -1.0;
    }

    return ((h - std::sqrt(discriminant)) / a);
}
```

Surface Normals and Multiple Objects

06

표면 법선과 다중 객체 — Hittable 추상화

1. Hittable 추상 클래스

Sphere 클래스를 만들기 전에, 충돌 가능한 객체들을 위한 추상화 클래스 Hittable을 만든다.

2. Hit 함수

Hittable을 상속받는 클래스는 광선을 받는 Hit 함수를 정의해야 한다.

3. HitRecord

충돌한 여러 객체들 중 하나를 선택해야 할 수 있으므로 충돌 정보를 기록해 둔다. 앞에서 HitSphere 함수로 알 수 있던 정보들(P, Normal, t)을 HitRecord에 기록한다.

```
class HitRecord
{
public:
    void SetFaceNormal(c

    // ...

    // ...

    // ...

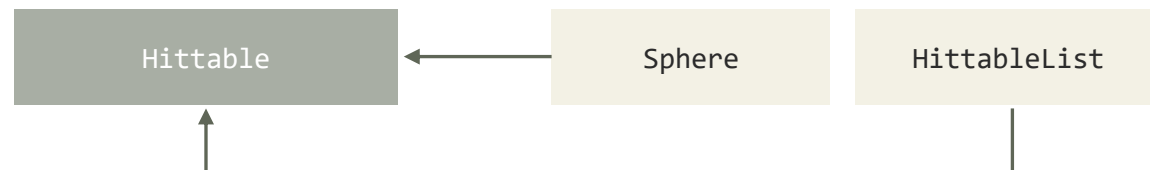
    Point3 P;
    Vec3 Normal;
    double T;
    bool bFrontFace;
};
```

HitRecord

```
class Hittable
{
public:
    virtual ~Hittable() = default;

    virtual bool Hit(
        const Ray& r,
        const Interval& rayT,
        HitRecord& hitRecord
    ) const = 0;
};
```

class Hittable



Sphere와 HittableList 모두 Hittable을 상속해 Hit()를 구현한다

Surface Normals and Multiple Objects

표면 법선과 다중 객체 — 유효 범위와 법선의 방향

```
if (discriminant < 0.0)
{
    return false;
}

auto squareRootDiscriminant = std::sqrt(discriminant);

// Find the nearest root that lies in the acceptable range
// 유효한 충돌 구간 추가
auto root = (h - squareRootDiscriminant) / a;
if (!rayT.Surrounds(root))
{
    root = (h + squareRootDiscriminant) / a;
    if (!rayT.Surrounds(root))
    {
        return false;
    }
}
```

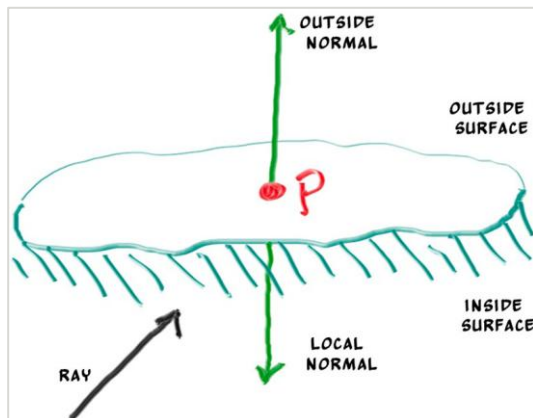
Sphere::Hit 함수

유효 범위

판별식이 음수일 경우 false.

작은 해를 먼저 검사(Surrounds 함수)하고, 그것이 충돌 구간을 벗어났다면 큰 해를 검사한다.

두 해 모두 범위를 벗어났다면 false를 반환한다.



법선의 방향

광선이 구의 외부에서 오는지 내부에서 오는지(외부 표면을 향하는지, 내부 표면을 향하는지) 판별해야 한다. 양면 종이처럼 각 면이 다르게 렌더링되거나, 유리공처럼 안과 밖이 있는 객체에서 중요하다.

법선과 광선을 내적해서 결정하는데, 법선이 항상 밖을 향하는지, 광선과 항상 반대인지에 따라 결정 방법이 나뉜다.

Surface Normals and Multiple Objects

06

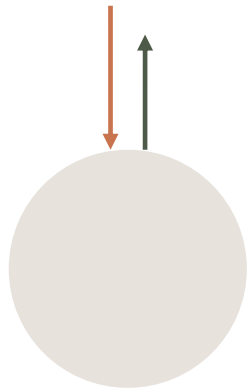
표면 법선과 다중 객체 — 앞면(front face) 판별

가정 1

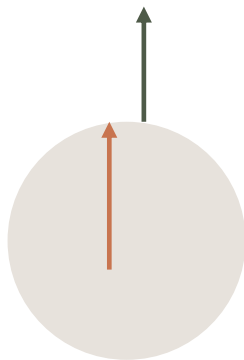
법선은 항상 밖을 향한다

우리가 구한 법선은 항상 구 중심에서 밖을 향한다. 그래서 광선이 구 외부에서 들어오면 법선과 광선은 방향이 반대다.

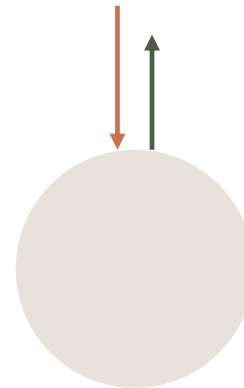
→ 내적 결과로 광선이 밖에서 오는지 안에서 나가는지를 결정할 수 있다.



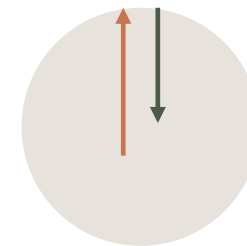
front face (dot < 0)



back face (dot > 0)



외부 → 내부 (dot < 0)



내부 → 외부 (dot < 0)

— 광선 — 법선

가정 2

법선은 항상 광선과 반대

광선이 구 외부에서 오면 법선은 밖을 향하고, 광선이 구 내부에서 나가면 법선은 반대로 안쪽을 향한다.

→ 내적 결과가 항상 음수이므로, 내적만으로는 밖에서 오는지 안에서 나가는지를 결정할 수 없다.

Surface Normals and Multiple Objects

06

표면 법선과 다중 객체 — SetFaceNormal

가정 2 채택

법선은 항상 광선과 반대 방향

내적의 결과를 이용해 광선의 방향(앞면인지)을 먼저 저장하고, 법선의 방향을 정해준다.

저장된 법선은 항상 광선과 반대이므로 나중에는 내적만으로 앞면·뒷면을 알 수 없다. 그래서 교차할 때 외향 법선과 내적인 결과를 bFrontFace에 따로 기록해 둔다.

결정 시점

이 결정을 색칠 시점에 할지 교차 시점에 할지 선택할 수 있는데, 이 책에서는 교차 시점(Hit)에서 결정된다.

```
void SetFaceNormal(const Ray& r, const Vec3& outwardNormal)
{
    // 히트 레코드 법선 벡터를 설정합니다.
    // 참고: 매개변수 'outwardNormal'은 단위 길이를 가진다고 가정합니다.

    bFrontFace = Dot(r.Direction(), outwardNormal) < 0;
    Normal = bFrontFace ? outwardNormal : -outwardNormal;
}
```

bFrontFace	Dot(광선 방향, 외향 법선) < 0 이면 광선이 밖에서 들어오는 앞면
Normal	앞면이면 외향 법선 그대로, 뒷면이면 뒤집어서 항상 광선과 반대 방향으로 저장

Surface Normals and Multiple Objects

표면 법선과 다중 객체 — HittableList

06

1. HittableList

Hittable 객체의 리스트를 저장하는 클래스를 만든다. 리스트는 `std::vector`를 사용하고, 각 Hittable 객체는 `std::shared_ptr`로 관리한다.

2. 왜 `shared_ptr`인가

여러 지오메트리가 머티리얼을 공유할 수 있고, 메모리 관리를 편하게 하기 위해서다.

3. 메인 헤더

자주 쓰이는 헤더, 상수, 유틸리티 함수를 포함하는 메인 헤더 파일이 추가되었다.

4. Interval

유효 범위를 표현하는 클래스가 추가되었다.

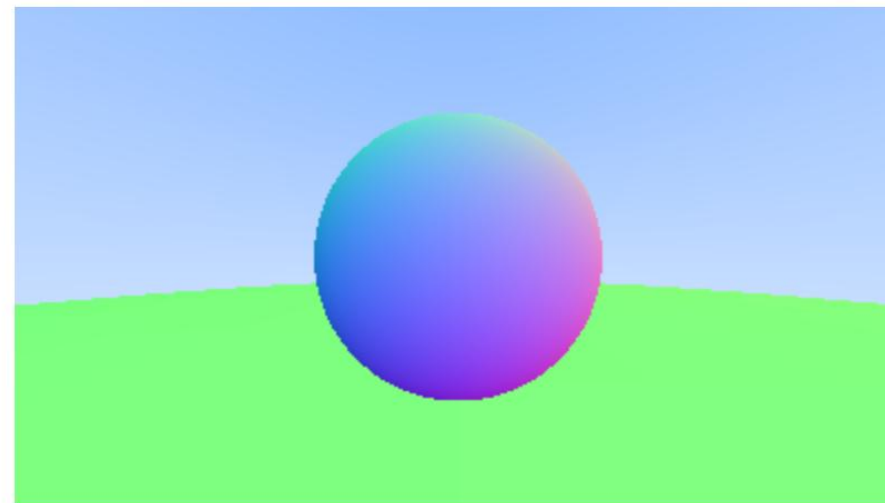
PPM Viewer

Choose a PPM image to view

파일 선택 normal.ppm

Reload image

File: normal.ppm



빌드 결과물 — 법선을 색으로 표현한 구와 지면

Moving Camera Code Into Its Own Class

카메라 코드를 자체 클래스로 이동

07

+102

추가된 줄

-74

삭제된 줄

74줄이 사라진 main 함수

카메라와 렌더링 관련 코드를 Camera 클래스로 옮겼다. main에는 월드 구성과 카메라 설정, Render 호출만 남는다.

Phase1.7.Moving Camera Code Into Its Own Class		
(카메라 코드를 자체 클래스로 이동)		
3 changed files		
Phase1Camera.h	3	3
Phase1Hittable.h	4	4
Phase1Kernel.cu	5	5

커밋 변경 내역

```
int main()
{
    HittableList world;
    world.Add(std::make_shared<Sphere>(Point3(0.0, 0.0, -1.0), 0.5));
    world.Add(std::make_shared<Sphere>(Point3(0.0, -100.5, -1.0), 100.0));

    // Camera
    Camera camera;

    camera.aspectRatio = 16.0 / 9.0;
    camera.imageWidth = 400;

    // Render

    camera.Render(world);

    return 0;
}
```

정리된 main 함수

Antialiasing

안티 앨리어싱 — 문제와 해결 방향

08

1. 앨리어싱 (계단 현상)

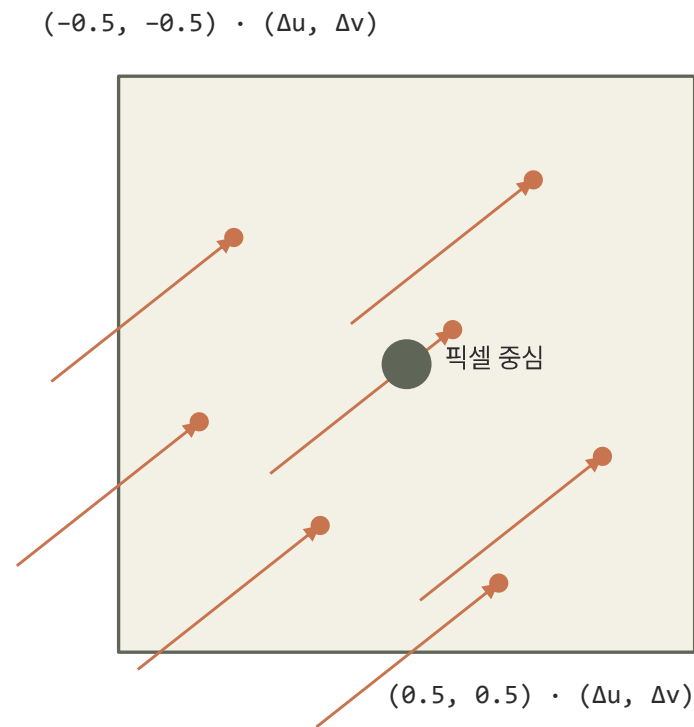
렌더링된 이미지를 확대해 보면 계단 현상이 보이는데, 이것을 앨리어싱 또는 재기 (jaggies)라고 부른다. 현실에서는 이미지가 연속적이고 자연스럽게 색이 섞여 보이는데 우리의 픽셀은 그렇지 못하다.

2. 점 샘플링

지금은 픽셀의 중앙을 통과하는 광선 하나만 쓰는 “점 샘플링”을 하고 있다. 검정과 흰색이 섞인 격자에 광선을 쏘았는데 검정 픽셀만 맞았다면, 회색으로 보여야 할 이미지가 검정으로만 보이게 된다(극단적인 예시).

3. 해결 — 주변을 샘플링

픽셀 주변의 광선들을 모아 샘플링하고 적분(평균)한다. 여기서 주변은 픽셀 중심 기준의 사각형으로 한정한다.



픽셀 중심 주변의 정사각형 안에서 무작위 위치로 광선을 여러 번 쏜다

Antialiasing

안티 앨리어싱 — 구현

08

```
for (int pixelIndex = 0; pixelIndex < imageWidth; pixelIndex++)
{
    Color pixelColor(0.0, 0.0, 0.0);

    for (int sampleIndex = 0; sampleIndex < samplesPerPixel; sampleIndex++)
    {
        Ray ray = GetRay(pixelIndex, scanlineIndex);
        pixelColor += RayColor(ray, world);
    }

    WriteColor(std::cout, mPixelSamplesScale * pixelColor);
}
```

```
void Initialize()
{
    mImageHeight = static_cast<int>(imageWidth / aspectRatio);
    mImageHeight = (mImageHeight < 1) ? 1 : mImageHeight;

    mPixelSamplesScale = 1.0 / static_cast<double>(samplesPerPixel);

    mCenter = Point3(0.0, 0.0, 0.0);
}
```

```
Ray GetRay(int pixelIndex, int scanlineIndex) const
{
    // Construct a camera ray originating from the origin and directed at randomly sampled
    // point around the pixel location pixelIndex, scanlineIndex.

    auto offset = SampleSquare();

    // 범위를 사각형으로 그려보면 크기 1짜리 정사각형이 나옴
    auto pixelSample =
        mPixel00Location
        + ((pixelIndex + offset.X()) * mPixelDeltaU)
        + ((scanlineIndex + offset.Y()) * mPixelDeltaV);

    auto rayOrigin = mCenter;
    auto rayDirection = pixelSample - rayOrigin;

    return Ray(rayOrigin, rayDirection);
}

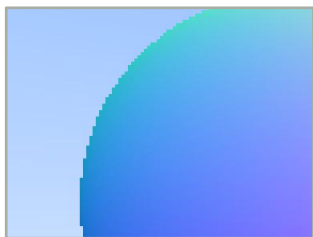
Vec3 SampleSquare() const
{
    // Returns the vector to a random point in the [-0.5, -0.5] - [+0.5, +0.5] unit square
    return Vec3(RandomDouble() - 0.5, RandomDouble() - 0.5, 0.0);
}
```

1. 정해진 횟수(samplesPerPixel)만큼 반복해서 광선을 모으고, pixelColor에 누적한다.
2. GetRay — 픽셀 중심 주변 [-0.5, 0.5] 정사각형 안의 무작위 점을 지나는 광선을 만든다.
3. 누적된 pixelColor에 mPixelSamplesScale (= 1 / samplesPerPixel)을 곱해 샘플링 횟수만큼 나눠준다.

Antialiasing

안티 앨리어싱 — 결과 비교

08



Before

점 샘플링 — 구의 경계가 계단처럼 보인다.

경계 확대



After

픽셀당 여러 샘플 — 경계의 색이 부드럽게 섞인다.

경계 확대

Diffuse Materials

확산 재질 — 무작위 반사

확산 객체

확산 객체는 자체적으로 빛을 방출하지 않는다. 주변의 색상을 띠지만, 고유한 색상으로 이를 조절한다. 확산 표면에서 반사되는 빛은 방향이 무작위화된다. 따라서 무작위로 반사되는 광선을 구현하는 것이 핵심이다. 처음에는 모든 방향으로 균등하게 반사시키는 표면을 구현한다.

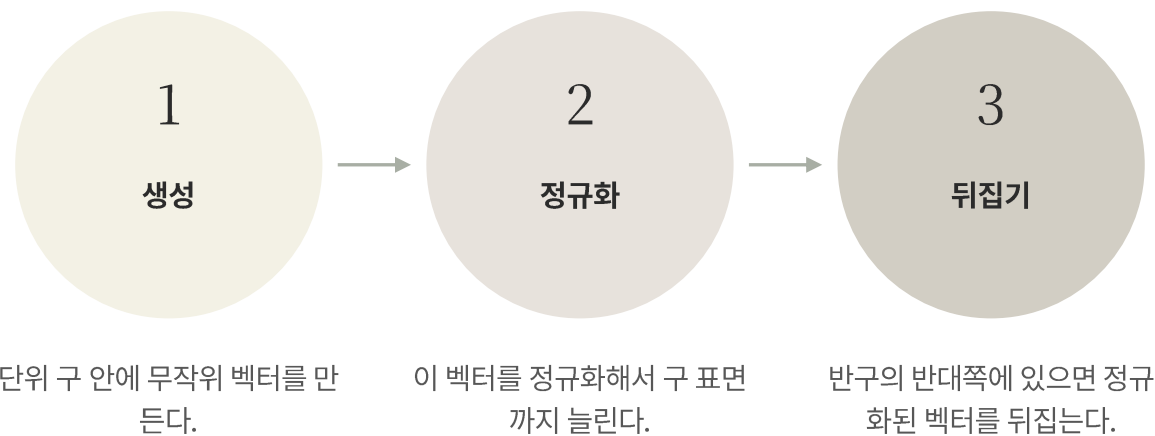
먼저 랜덤한 벡터를 생성하는 유틸리티 함수를 정의한다.

```
static Vector3 Random()
{
    return Vector3(RandomDouble(), RandomDouble(), RandomDouble());
}

static Vector3 Random(double minimum, double maximum)
{
    return Vector3(
        RandomDouble(minimum, maximum),
        RandomDouble(minimum, maximum),
        RandomDouble(minimum, maximum)
    );
}
```

기각 방법 (Rejection Method)

완전 반대 방향으로 가지 않도록 반구 표면에 있는 벡터만 얻는 방법이 필요하다. 복잡한 방법 대신, 원하는 벡터가 나올 때까지 랜덤으로 계속 생성한다.

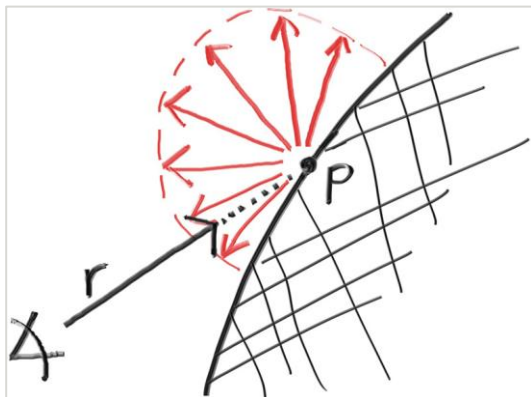


Diffuse Materials

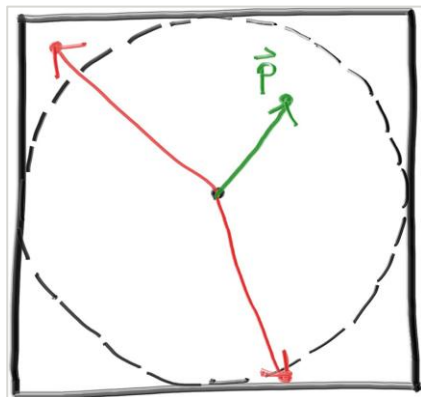
확산 재질 — 단위 구에서 무작위 벡터 얻기

09

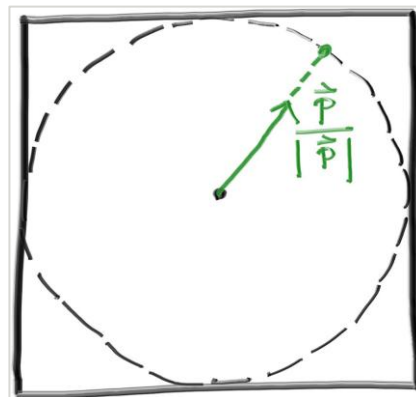
단위 구가 나온 이유는 균등하게 확산하기 위해서다. 구 표면 위에 새로운 반구를 상상해 보면 되겠다.



① 표면 위의 반구와 단위 구



② 원 안에 있는 점을 고른다



③ 정규화해서 단위 길이로 연장

단위 구라고 했으니 원 안에 있는 점을 골라 정규화하고, 단위 길이 벡터로 만들어 연장하면 완성이다.

```
inline Vector3 RandomUnitVector()
{
    while (true)
    {
        auto p = Vector3::Random(-1.0, 1.0);
        auto lengthSquared = p.LengthSquared();

        // 매우 작은값이 제공되면 언더플로워되어 0이 될 수 있다.
        if (1e-160 < lengthSquared && lengthSquared <= 1.0)
        {
            return p / std::sqrt(lengthSquared);
        }
    }
}
```

RandomUnitVector

1. 단위 정육면체 안에서 무작위 점 p 를 뽑고, 단위 구 안($\text{lengthSquared} \leq 1$)에 들 때까지 반복한다.
2. * 매우 작은 값이 제공되면 언더플로워되어 0이 될 수 있어, $1e-160$ 보다 큰 경우만 사용한 뒤 길이로 나눠 정규화한다.

Diffuse Materials

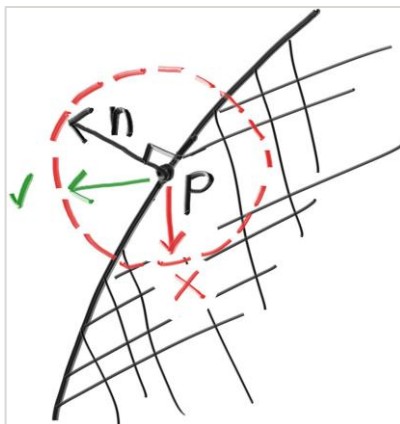
확산 재질 — 반구 위로 반사하기

09

```
inline Vector3 RandomOnHemisphere(const Vector3& normal)
{
    Vector3 unitSphereDirection = RandomUnitVector();

    // In the same hemisphere as the normal
    if (Dot(unitSphereDirection, normal) > 0.0)
    {
        return unitSphereDirection;
    }

    return -unitSphereDirection;
}
```



RandomOnHemisphere

법선과 내적해서 음수일 경우(반구의 반대쪽에 있음) 벡터를 반대로 뒤집는다.

```
Color RayColor(const Ray& ray, int depth, const Hittable& world) const
{
    // If we've exceeded the ray bounce limit, no more light is gathered
    if (depth <= 0)
    {
        return Color(0.0, 0.0, 0.0);
    }

    HitRecord hitRecord;

    if (world.Hit(ray, Interval(0.001, Infinity), hitRecord))
    {
        // 랜덤하게 광선을 보낸다.
        Vec3 direction = RandomOnHemisphere(hitRecord.Normal);
        // ...

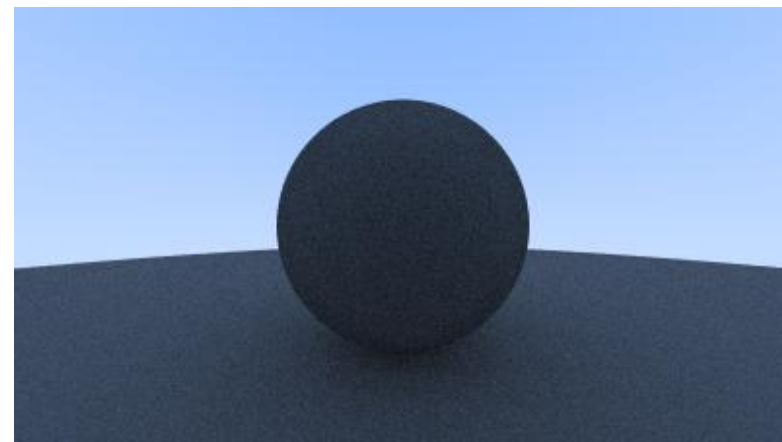
        return 0.5 * RayColor(Ray(hitRecord.P, direction), depth - 1, world);
    }

    Vector3 unitDirection = UnitVector(ray.Direction());
    auto a = 0.5 * (unitDirection.Y() + 1.0);

    return (1.0 - a) * Color(1.0, 1.0, 1.0) + a * Color(0.5, 0.7, 1.0);
}
```

재귀 RayColor

재귀 함수다. 반사된 광선에 대해서 또 계산을 한다. 반사될 때마다 절반(0.5)씩 감소한다.
반사되는 색상을 100% 유지하면 흰색, 0% 유지하면 검은색이다.



첫 번째 확산 재질 렌더

Diffuse Materials

확산 재질 — 재귀 깊이 제한

09

```
Color RayColor(const Ray& ray, int depth, const Hittable& world) const
{
    // If we've exceeded the ray bounce limit, no more light is gathered
    if (depth <= 0)
    {
        return Color(0.0, 0.0, 0.0);
    }

    HitRecord hitRecord;

    // ...
    if (world.Hit(ray, Interval(0.001, Infinity), hitRecord))
    {
        // 랜덤하게 광선을 보낸다.
        Vec3 direction = RandomOnHemisphere(hitRecord.Normal);
        // ...

        return 0.5 * RayColor(Ray(hitRecord.P, direction), depth - 1, world);
    }

    Vector3 unitDirection = UnitVector(ray.Direction());
    auto a = 0.5 * (unitDirection.Y() + 1.0);

    return (1.0 - a) * Color(1.0, 1.0, 1.0) + a * Color(0.5, 0.7, 1.0);
}
```

RayColor(ray, depth, world) — 호출할 때마다 depth - 1

1. 스택 오버플로우

함수가 멈추지 않고 계속 호출된다면 스택 오버플로우가 될 것이다.

2. maxDepth

Camera 클래스에 depth(maxDepth)를 추가해 제한을 건다. depth가 0 이하가 되면 더 이상 빛을 모으지 않고 검정을 반환한다.

```
class Camera
{
public:
    double aspectRatio = 1.0;           // Ratio of image width over height
    int imageWidth = 100;                // Rendered image width in pixel count
    int samplesPerPixel = 10;            // Count of random samples for each pixel
    int maxDepth = 10;                   // Maximum number of ray bounces into scene
};
```

class Camera —
maxDepth 추가

```
for (int pixelIndex = 0; pixelIndex < imageWidth; pixelIndex++)
{
    Color pixelColor(0.0, 0.0, 0.0);

    for (int sampleIndex = 0; sampleIndex < samplesPerPixel; sampleIndex++)
    {
        Ray ray = GetRay(pixelIndex, scanlineIndex);
        pixelColor += RayColor(ray, maxDepth, world);
    }

    WriteColor(std::cout, mPixelSamplesScale * pixelColor);
}
```

Camera::Render

Diffuse Materials

확산 재질 — 그림자 여드름 (Shadow Acne)

09

1. 부동소수점 오차

광선과 표면의 교차점을 계산할 때 부동소수점 오차가 생길 가능성은 매우 크다.

2. 같은 표면과 다시 교차

오차로 인해 원래 교차점보다 아래에 있다면 다시 그 표면과 교차할 수 있다. 그게 반복되면 검정색이 된다. 이를 그림자 여드름(shadow acne)이라고도 부른다.

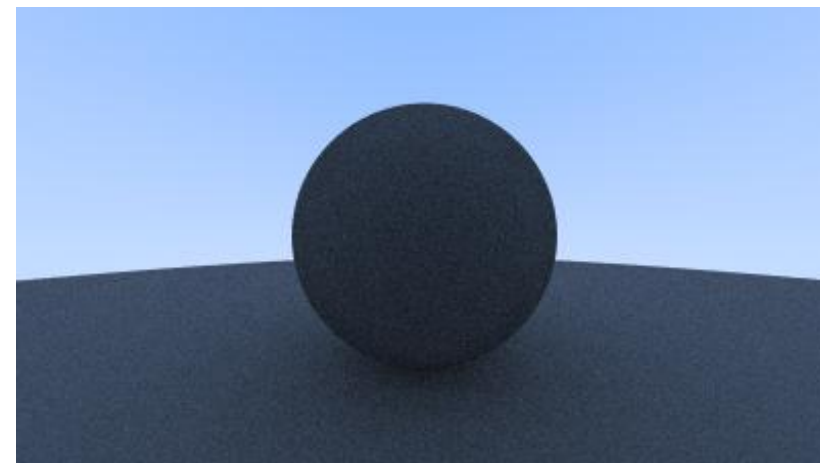
3. 너무 작은 t는 무시

앞에서 만든 유효 범위(Interval)를 이용해서 너무 작은 값은 무시하도록 제한을 건다.

```
Interval(0.001, Infinity)
```

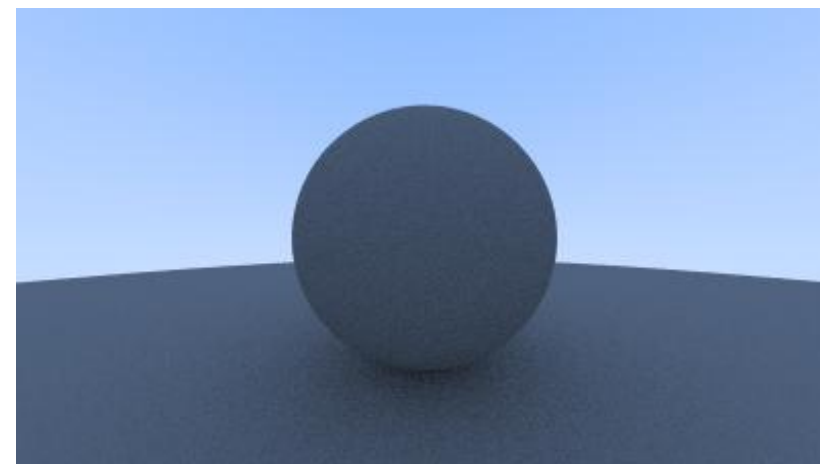
Before

표면이 다시
자기 자신과 교차



After

Interval(0.001, ∞)



Diffuse Materials

확산 재질 — 랬베르시안(Lambertian) 분포

이미 반구에 걸쳐 균일하게 광선을 산란시켰지만 더 나은 방법이 있다. 랬베르시안(Lambertian) 분포다.

1. 접하는 두 단위 구

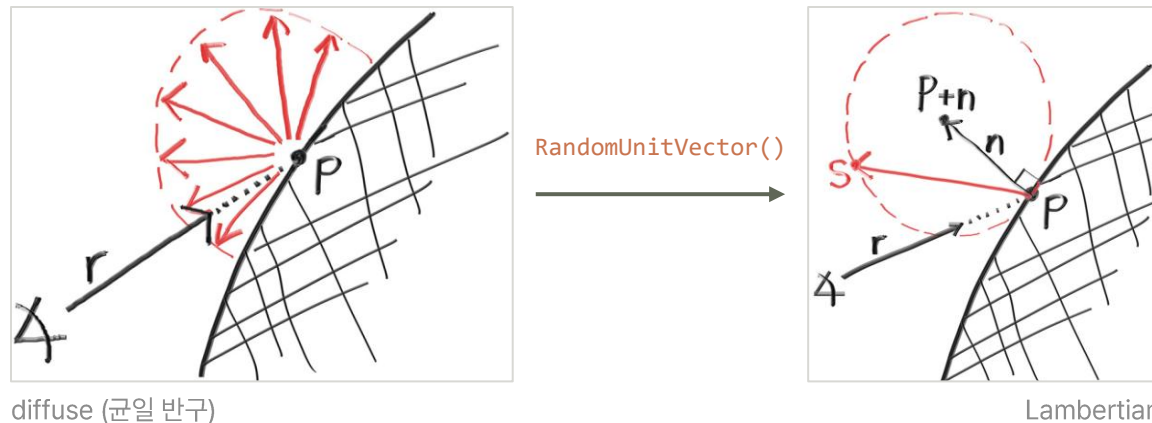
표면의 교차점 P 와 표면 법선 n 이 있다. 표면과 딱 접하는 단위 반지름 구를 2개 그릴 수 있는데, 하나는 중심이 $P + n$ 이고 하나는 중심이 $P - n$ 이다.

2. 무작위 점 S

충돌 지점 P 에서 무작위 점 S 로 광선을 보낼 건데, 이 방향은 $S - P$ 다.

3. 법선 + 랜덤 단위 벡터

$S - P$ 는 벡터 성질을 이용해 노말과 랜덤 단위 길이 벡터의 덧셈으로 바꿀 수 있다.



```
// Hit 범위의 최소값을 줌 부동소수점 오차로 인해 똑같은 위치로
// 반사되는 것을 방지함
if (world.Hit(ray, Interval(0.001, Infinity), hitRecord))
{
    // 랜덤하게 광선을 보낸다.
    //Vec3 direction = RandomOnHemisphere(hitRecord.Normal);
    // 충돌 지점 P에서 무작위 점 S로 광선을 보낸다.
    Vec3 direction = hitRecord.Normal + RandomUnitVector();

    return 0.5 * RayColor(Ray(hitRecord.P, direction), depth - 1, world);
}

Vector3 unitDirection = UnitVector(ray.Direction());
auto a = 0.5 * (unitDirection.Y() + 1.0);

return (1.0 - a) * Color(1.0, 1.0, 1.0) + a * Color(0.5, 0.7, 1.0);
```

최종적으로 코드는 왼쪽처럼 된다.

direction =
Normal +
RandomUnitVector()

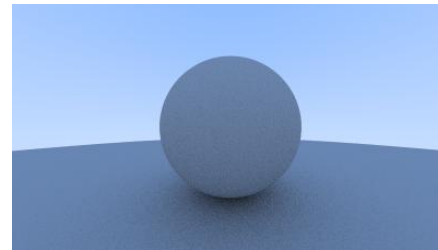
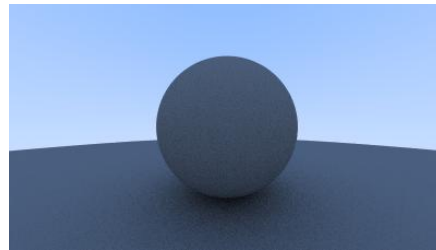
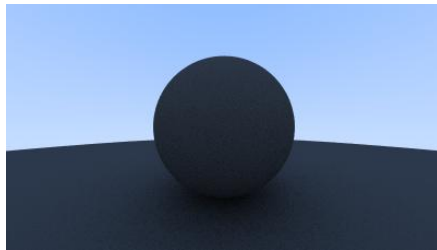
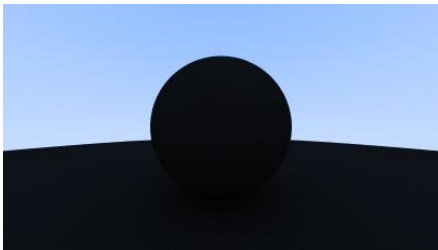
Diffuse Materials

확산 재질 — 반사율에 따른 결과

09

RayColor에서 반사될 때 유지하는 비율(0.5)을 바꿔 본다. 값이 클수록 더 많은 빛을 반사해 밝아진다.

```
return 0.5 * RayColor(Ray(hitRecord.P, direction), depth - 1, world);
```



반사율 0.1 → 0.9 · 원의 명도는 반사율을 뜻한다

Diffuse Materials

확산 재질 — 감마 보정

09

1. 선형 공간 vs 감마 공간

이미지 편집 프로그램은 이미지가 감마 보정되었다고 생각하는데, 우리는 감마 보정이 안 된 선형 공간의 이미지를 생성했다.

2. LinearToGamma

따라서 선형 공간에서 감마 공간으로 변환해 준다. 감마 2를 쓰므로 각 성분에 제곱근을 취한다.

```
inline double LinearToGamma(double linearComponent)
{
    if (linearComponent > 0.0)
    {
        return std::sqrt(linearComponent);
    }
    return 0.0;
}
```

```
void WriteColor(std::ostream& out, const Color& pixelColor)
{
    auto r = pixelColor.X();
    auto g = pixelColor.Y();
    auto b = pixelColor.Z();

    // Apply a linear to gamma transform for gamma = 2.0
    r = LinearToGamma(r);
    g = LinearToGamma(g);
    b = LinearToGamma(b);

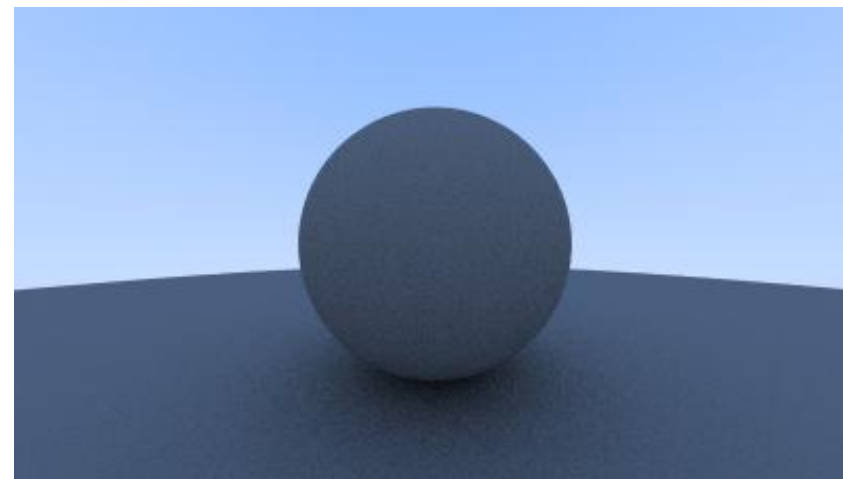
    // [0,1] 범위의 컴포넌트 값을 바이트 범위 [0,255]로 변환합니다.
    static const Interval intensity(0.000, 0.999);

    int rByte = static_cast<int>(256 * intensity.Clamp(r));
    int gByte = static_cast<int>(256 * intensity.Clamp(g));
    int bByte = static_cast<int>(256 * intensity.Clamp(b));

    // 픽셀 색상 컴포넌트를 출력합니다.
    out << rByte << ' ' << gByte << ' ' << bByte << '\n';
}
```

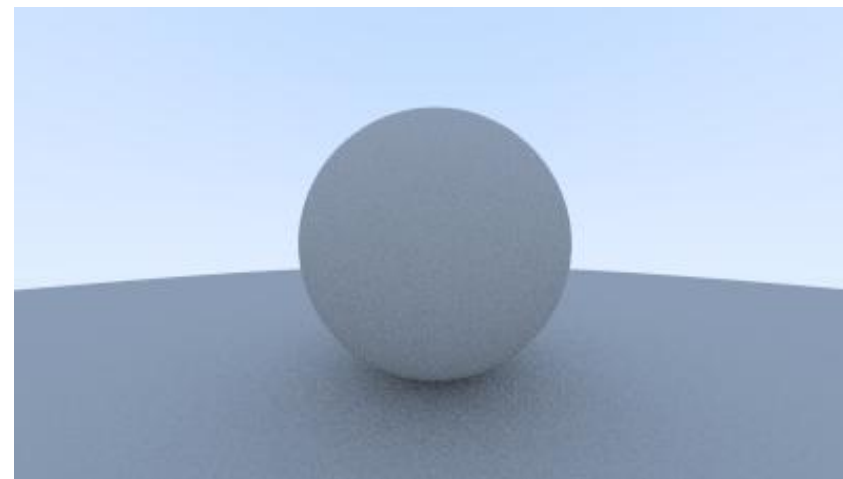
보정 전

반사율 0.5



보정 후

반사율 0.5



Diffuse Materials

확산 재질 — 감마 보정 전 · 후 비교

09

0.1

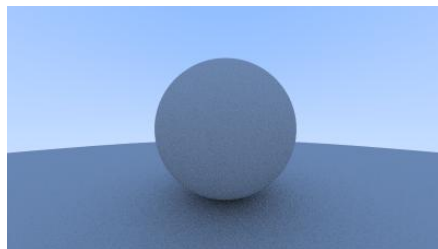
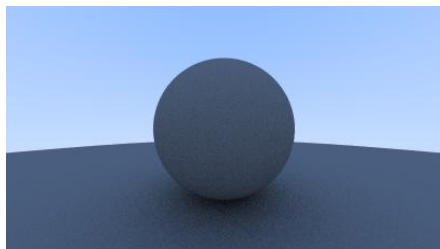
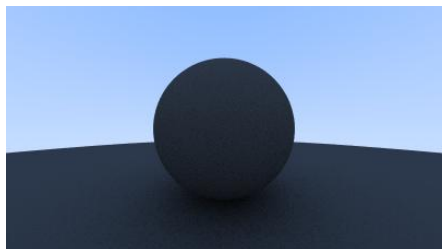
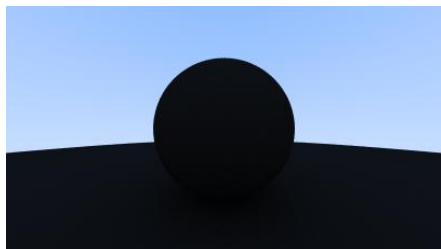
0.3

0.5

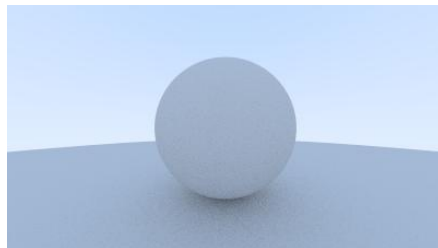
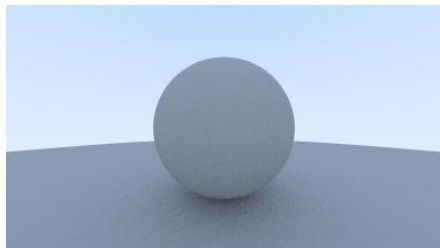
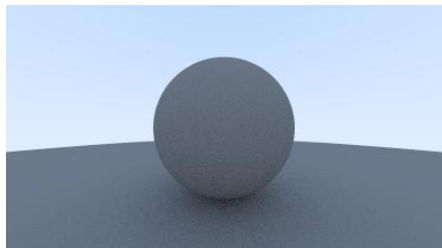
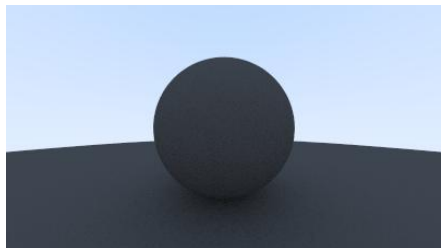
0.7

0.9

선형 공간 보정 전



감마 공간 보정 후



감마 보정 후에는 반사율에 따라 어두운 쪽에서 밝은 쪽으로 밝기가 훨씬 고르게 변한다.

Metal

금속 — 재질 클래스 설계

10

1. Material 추상 클래스

다양한 재질을 위해 고유한 동작을 캡슐화하는 추상 재질 클래스를 만든다. 산란된 광선을 생성하고, 산란되었을 때 광선이 얼마만큼 감쇠해야 하는지 알려준다.

```
class Material
{
public:
    virtual ~Material() = default;
    virtual bool Scatter(const Ray& rayIn, const HitRecord& hitRecord, Color& attenuation, Ray& scattered) const
    {
        return false;
    }
};
```

2. HitRecord에 재질 기록

RayColor에서 재질을 참조할 수 있도록 한다. 이전에는 재질과 상관없이 벡터를 랜덤하게 보냈지만, 이제 재질에 따라 다르게 산란할 수 있다.

```
if (world.Hit(ray, Interval(0.001, Infinity), hitRecord))
{
    Ray scattered;
    Color attenuation;

    if (hitRecord.material->Scatter(ray, hitRecord, attenuation, scattered))
    {
        return attenuation * RayColor(scattered, depth - 1, world);
    }

    return Color(0.0, 0.0, 0.0);
}
```

3. Lambertian과 Metal

두 재질이 추가되었다. albedo는 재질의 색상이다. Lambertian 반사율은 빛이 감쇠될 수도 있는데, 여기서는 감쇠 없이 항상 산란하도록 구현했다.

```
bool Scatter(const Ray& rayIn, const HitRecord& hitRecord, Color& attenuation, Ray& scattered) const override
{
    Vec3 scatterDirection = hitRecord.Normal + RandomUnitVector();

    // Catch degenerate scatter direction
    if (scatterDirection.NearZero())
    {
        scatterDirection = hitRecord.Normal;
    }

    scattered = Ray(hitRecord.P, scatterDirection);
    attenuation = mAlbedo;

    return true;
}
```

4. NearZero

산란 벡터가 법선과 정확히 반대면 두 벡터의 합은 0이 되어 무한대나 NaN 같은 문제를 일으킬 수 있다. 0에 가까운지 검사하는 헬퍼 함수로 방지한다.

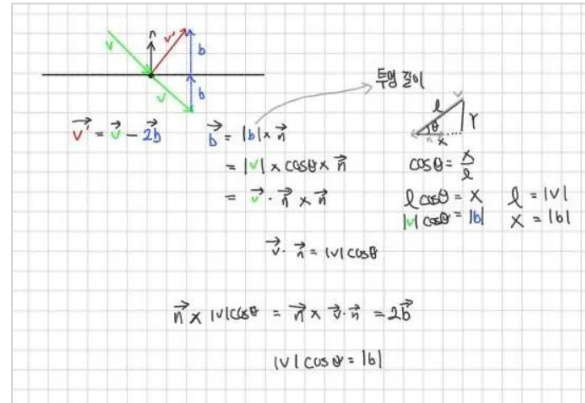
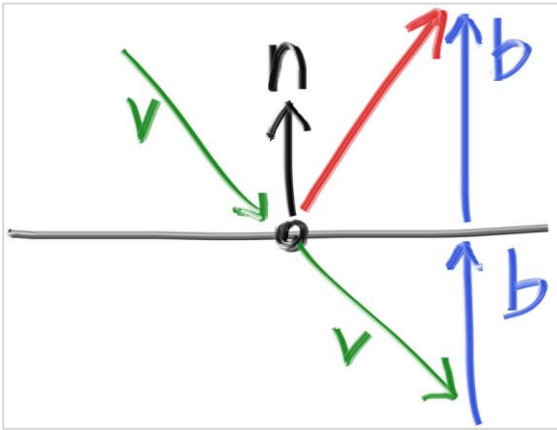
```
bool NearZero() const
{
    // Return true if the vector is close to zero in all dimensions
    auto threshold = 1e-8;

    return (std::fabs(E[0]) < threshold)
        && (std::fabs(E[1]) < threshold)
        && (std::fabs(E[2]) < threshold);
}
```

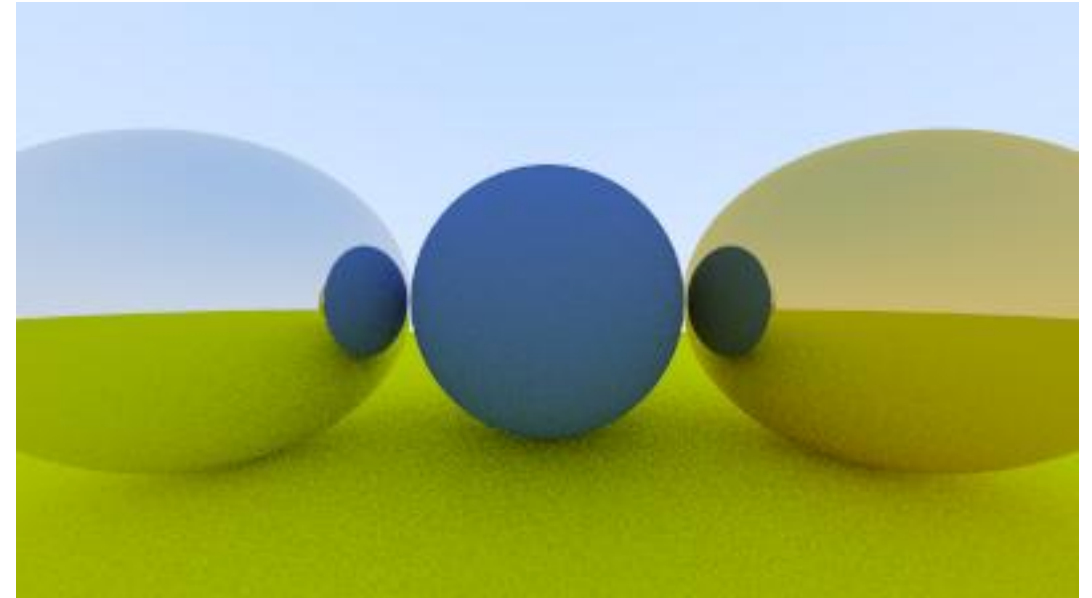
Metal

금속 — 정반사

금속은 랜덤하게 반사되는 게 아니라, 거울에 반사되는 빛처럼 입사각과 반사각이 같은 정반사를 한다.



1. 빨간색 벡터가 산란(scattered) 벡터다. 방향은 $v + 2b$ 다.
2. 벡터 b 를 구하려면 법선 벡터(방향)에 v 의 n 방향 투영 길이(크기)를 곱한다.
3. 벡터 v 와 b 는 방향이 반대이므로 투영 길이에 음수를 취해야 한다.



빌드 결과 — 양옆이 금속 구

$$\text{Reflect}(v, n) = v - 2 (v \cdot n) n$$

Metal

금속 — 퍼지 반사 (Fuzzy Reflection)

10

1. 퍼지 매개변수

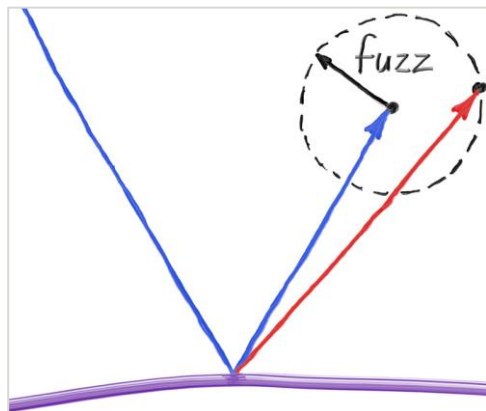
퍼지 매개변수를 추가해서 반사를 흐릿하게 할 수 있다.

2. 무작위 점 선택

반사 벡터 끝점에서 퍼지 값만큼의 반지름을 가진 구를 그리고, 그 구의 표면에서 랜덤한 점을 선택한다.

3. 흐림 정도

퍼지 값이 커질수록 반사가 더 흐릿해진다.

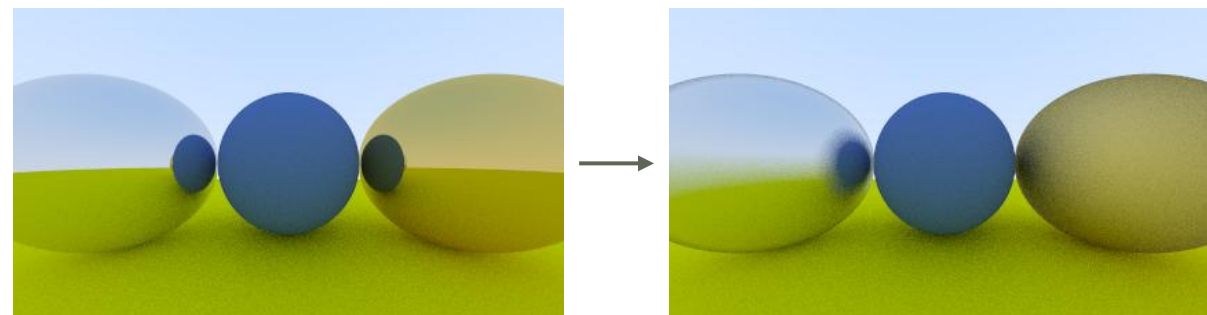


```
bool Scatter(const Ray& rayIn, const HitRecord& hitRecord, Color& attenuation, Ray& scattered) const override
{
    Vec3 reflected = Reflect(rayIn.Direction(), hitRecord.Normal);
    reflected = UnitVector(reflected) + (mFuzz * RandomUnitVector());

    scattered = Ray(hitRecord.P, reflected);
    attenuation = mAlbedo;

    return (Dot(scattered.Direction(), hitRecord.Normal) > 0);
}
```

```
auto materialLeft = std::make_shared<Metal>(Color(0.8, 0.8, 0.8), 0.3);
auto materialRight = std::make_shared<Metal>(Color(0.8, 0.6, 0.2), 1.0);
```



fuzz 없음

fuzz 왼쪽 0.3 · 오른쪽 1.0

Dielectrics

유전체 — 굴절과 스넬의 법칙

11

1. 유전체

새로운 재질인 유전체가 추가된다. 유전체란 물, 유리, 다이아몬드와 같은 투명한 재질이다. 광선이 유전체에 부딪히면 반사 광선과 굴절 광선으로 나뉜다.

2. 굴절률

굴절 광선이 굽어지는 정도인데, 재질에 따라 다르다.

3. 스넬의 법칙

굴절은 스넬의 법칙으로 계산한다. 증명을 이해하기보다 식을 그대로 코드에 적용한다고 생각한다.

$$\eta \cdot \sin \theta = \eta' \cdot \sin \theta'$$

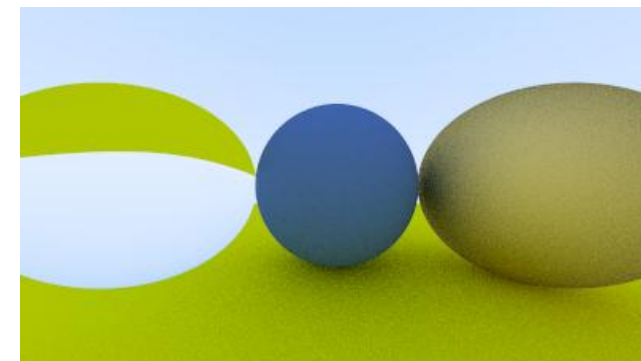
```
inline Vector3 Refract(const Vector3& uv, const Vec3& n, double etaInOverEtaOut)
{
    const double cosTheta = std::fmin(Dot(-uv, n), 1.0);

    const Vector3 refractPerpendicular = etaInOverEtaOut * (uv + cosTheta * n);
    const Vector3 refractParallel =
        -std::sqrt(std::fabs(1.0 - refractPerpendicular.LengthSquared())) * n;

    return refractPerpendicular + refractParallel;
}
```

$$\mathbf{R}'_{\perp} = \frac{\eta}{\eta'}(\mathbf{R} + (-\mathbf{R} \cdot \mathbf{n})\mathbf{n})$$

$$\mathbf{R}'_{\parallel} = -\sqrt{1 - |\mathbf{R}'_{\perp}|^2}\mathbf{n}$$



항상 굴절만 하도록 하면 이렇게 된다

Dielectrics

유전체 — 전반사

11

1. 해가 없는 광선

스넬의 법칙으로 해결할 수 없는 광선도 있다. 굴절각이 90° 를 넘어야 하는 경우 $\sin \theta'$ 가 1보다 커져야 하므로 식이 성립하지 않는다. 예) 유리(1.5) $\rightarrow$ 공기(1.0): $1.5 \cdot \sin \theta > 1.0$

2. 전반사

이때는 광선을 굴절시키지 않고 반사하도록 하자.

3. 공기 방울로 확인

결과가 이전과 다르지 않은데, 공기보다 큰 굴절률을 가진 구에서는 광선이 나가는 지점 어디에서도 전반사를 일으킬 입사 각도가 없다. 그래서 물 안에 있는 공기 방울을 표현하도록 굴절률을 조절한다.

있습니다. 스넬의 법칙과 $\sin \theta'$ 의 유도로 돌아가면:

$$\sin \theta' = \eta \eta' \cdot \sin \theta$$

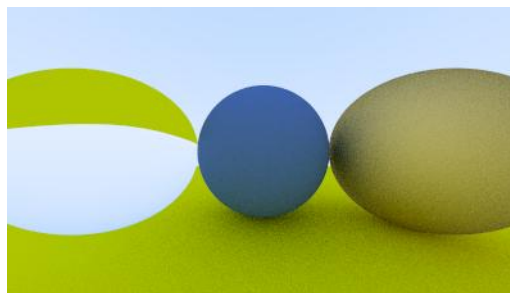
광선이 유리 내부에 있고 외부가 공기인 경우($\eta=1.5$, $\eta'=1.0$):

$$\sin \theta' = 1.5 \cdot 1.0 \cdot \sin \theta$$

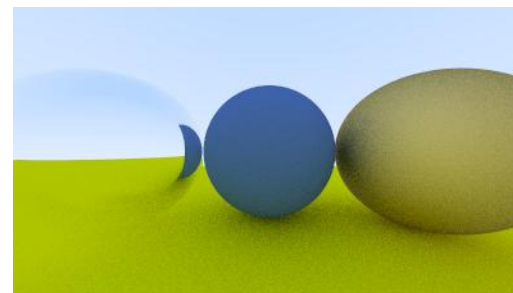
$\sin \theta'$ 의 값은 1보다 클 수 없습니다. 따라서 다음과 같다면:

$$1.5 \cdot \sin \theta > 1.0$$

```
const bool cannotRefract =  
    refractionRatio * sinTheta > 1.0;  
  
Vector3 direction;  
  
if (cannotRefract || Reflectance(cosTh  
{  
    direction = Reflect(unitDirection,
```



굴절률 1.5 — 이전과 동일



굴절률 1.00 / 1.33 — 물 속 공기 방울

```
auto materialLeft = make_shared<Dielectric>(1.00 / 1.33);
```

Dielectrics

유전체 — Schlick 근사와 속이 빈 유리구

11

1. Schlick 근사

실제 유리는 각도에 따라 반사율이 달라지는데, 이것을 구현하기 위해 Schlick의 반사율 근사를 사용한다.

```
static double Reflectance(double cosine, double refractionIndex)
{
    // Schlick의 반사율 근사 사용
    auto r0 = (1.0 - refractionIndex) / (1.0 + refractionIndex);
    r0 = r0 * r0;
    return r0 + (1.0 - r0) * std::pow((1.0 - cosine), 5);
}
```

2. 속이 빈 유리구

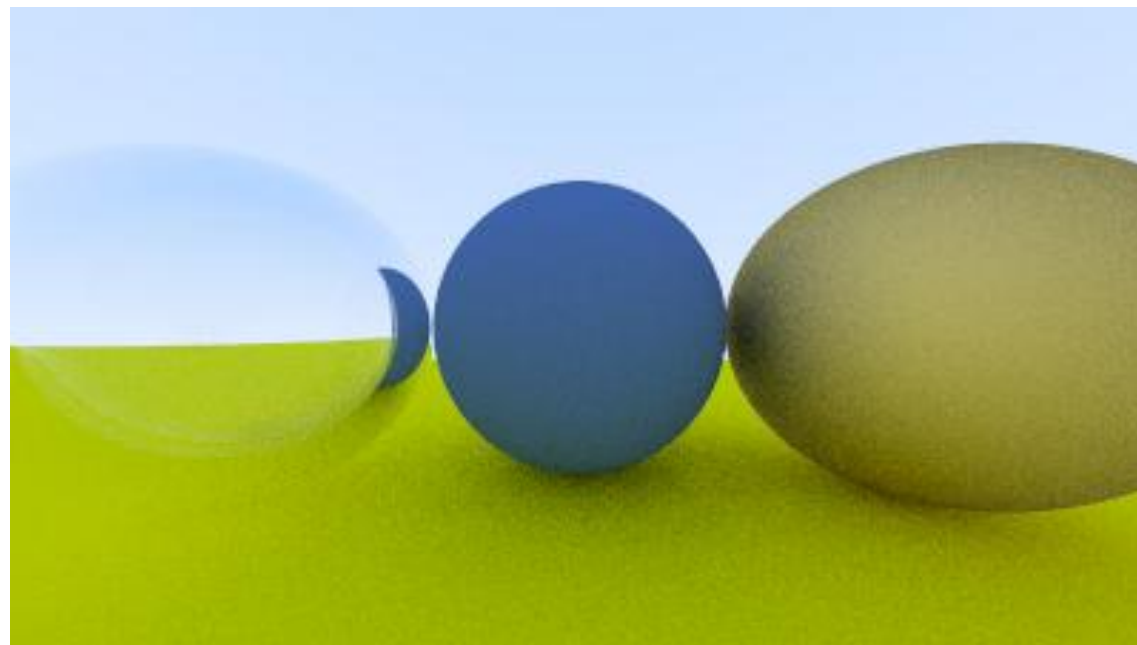
왼쪽에 있는 구를 바깥은 유리(1.50), 안쪽은 공기(1.00 / 1.50)로 모델링한다.

```
HittableList world;

auto materialGround = std::make_shared<Lambertian>(Color(0.8, 0.8, 0.0));
auto materialCenter = std::make_shared<Lambertian>(Color(0.1, 0.2, 0.5));
auto materialLeft = std::make_shared<Dielectric>(1.50);
auto materialBubble = std::make_shared<Dielectric>(1.00 / 1.50);
auto materialRight = std::make_shared<Metal>(Color(0.8, 0.6, 0.2), 1.0);

world.Add(std::make_shared<Sphere>(Point3(0.0, -100.5, -1.0), 100.0, materialGround));
world.Add(std::make_shared<Sphere>(Point3(0.0, 0.0, -1.2), 0.5, materialCenter));

world.Add(std::make_shared<Sphere>(Point3(-1.0, 0.0, -1.0), 0.5, materialLeft));
world.Add(std::make_shared<Sphere>(Point3(-1.0, 0.0, -1.0), 0.4, materialBubble));
world.Add(std::make_shared<Sphere>(Point3(1.0, 0.0, -1.0), 0.5, materialRight));
```



왼쪽 구가 속이 빈 유리구 — 바깥은 유리, 안쪽은 공기

Positionable Camera

위치 조정 가능한 카메라 — 시야각 (fov)

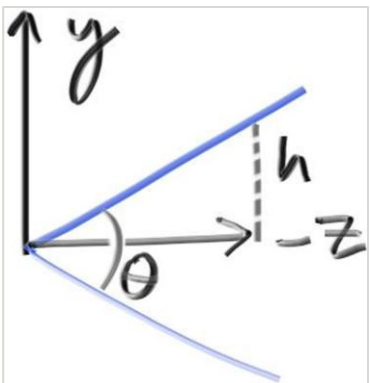
1. vFov 기준 뷰포트

뷰포트는 이제 수직 시야각(vFov) 기준으로 계산된다.

2. 더 넓은 세상

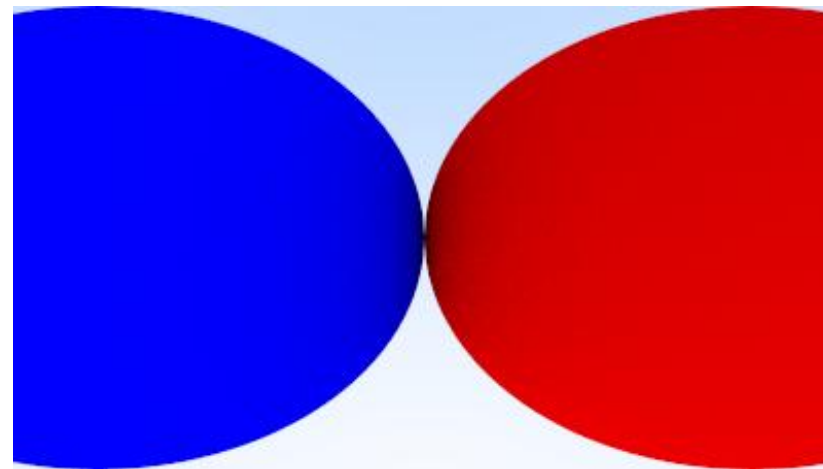
fov가 커지면 더 넓은 세상을 보게 된다.

```
// Determine viewport dimensions
auto focalLength = (lookFrom - lookAt).Length();
auto theta = DegreesToRadians(vFov);
auto h = std::tan(theta / 2);
auto viewportHeight = 2 * h * focalLength;
auto viewportWidth = viewportHeight * (static_cast<double>(imageWidth) / mImageHeight);
```



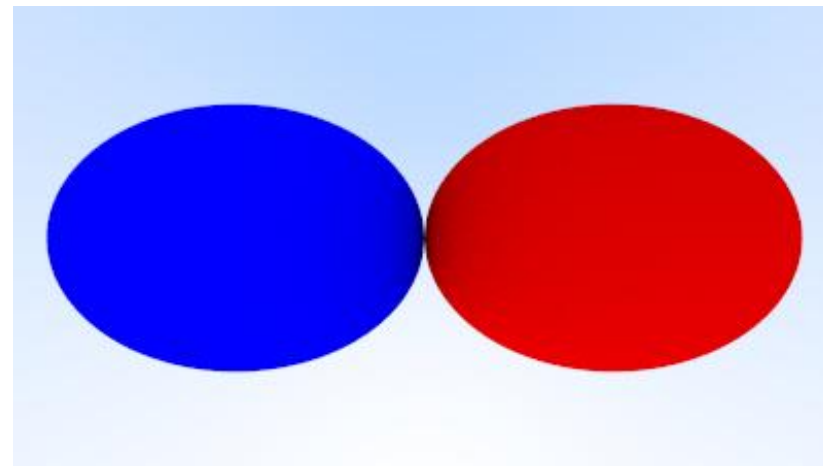
$$h = \tan(\theta / 2)$$
$$\text{viewportHeight} = 2 \cdot h \cdot \text{focalLength}$$

fov 90°



fov 120°

더 넓게 보인다



Positionable Camera

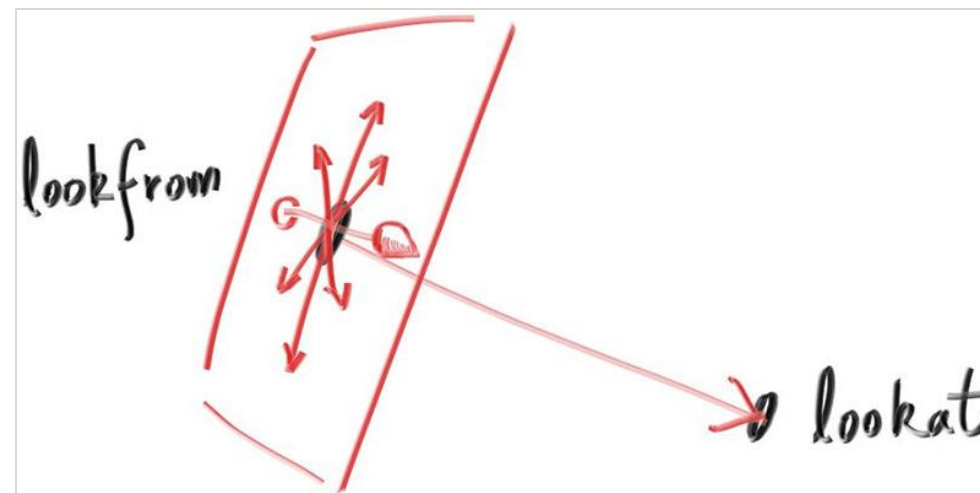
위치 조정 가능한 카메라 — look_from, look_at, $u \cdot v \cdot w$

look_from	카메라의 위치
look_at	카메라가 바라보는 점
view_up	카메라의 위쪽을 가리키는 벡터(수직이 아닐 수도 있다). 뷰 방향과 평행하지만 않다면 어떤 벡터든 지정할 수 있다. 월드 Y축 (0, 1, 0)을 사용하지만 꼭 그럴 필요는 없다.
u	카메라 오른쪽 단위 벡터
v	카메라 위쪽 단위 벡터
w	뷰 방향의 반대를 가리키는 단위 벡터

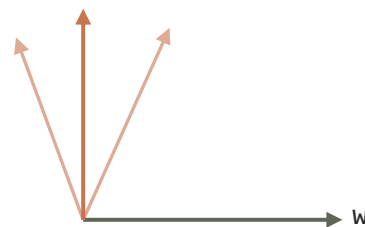
u , v , w 는 서로 직교한다(정규 직교 기저). 지금까지 카메라가 $-Z$ 방향을 바라봤다면, 임의의 뷰 카메라는 $-w$ 방향을 바라본다.

```
// 카메라 좌표 프레임에 대한 u,v,w 단위 기저 벡터 계산
w = UnitVector(lookFrom - lookAt);
u = UnitVector(Cross(vUp, w));
v = Cross(w, u);
```

이제 카메라 시점을
바꿔보자 →

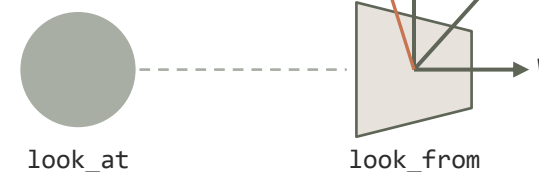


view_up



어떤 view_up을 써도 외적하면
수직인 벡터가 나온다

view_up

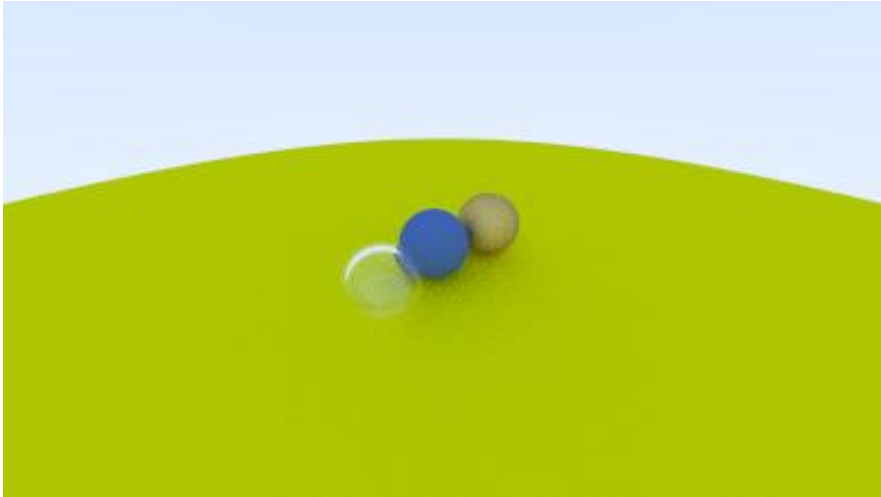


Positionable Camera

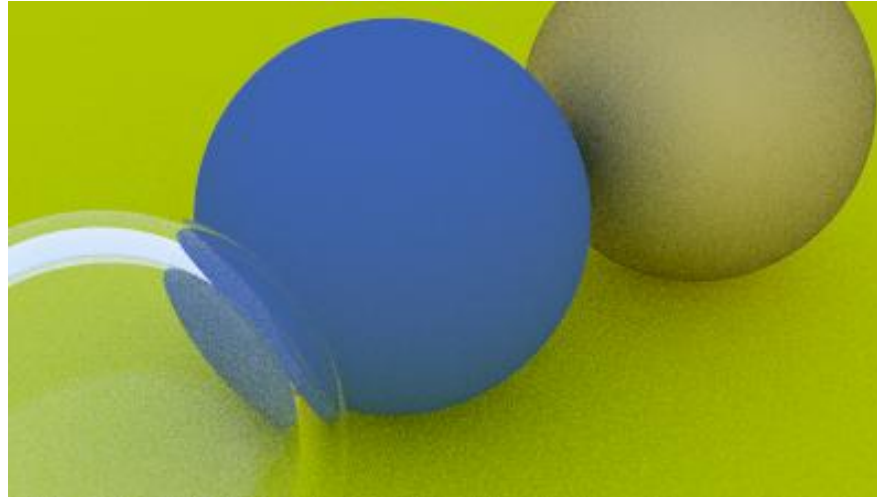
위치 조정 가능한 카메라 — 시점 바꾸기

12

lookFrom (-2, 2, 1)에서 lookAt (0, 0, -1)을 바라보도록 시점을 바꾼 결과. 시야각을 20°로 좁히면 확대한 것처럼 보인다.



vFov 90°



vFov 20°

```
// Camera
Camera camera;

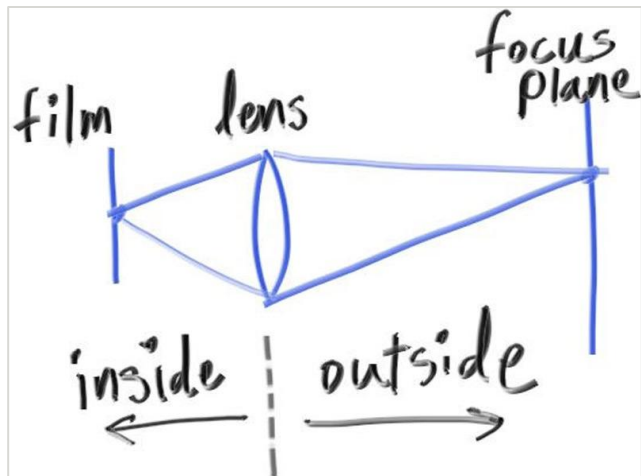
camera.aspectRatio = 16.0 / 9.0;
camera.imageWidth = 400;
camera.samplesPerPixel = 100;
camera.maxDepth = 50;

camera.vFov = 90;
camera.lookFrom = Point3(-2, 2, 1);
camera.lookAt = Point3(0, 0, -1);
camera.vUp = Vec3(0, 1, 0);
```

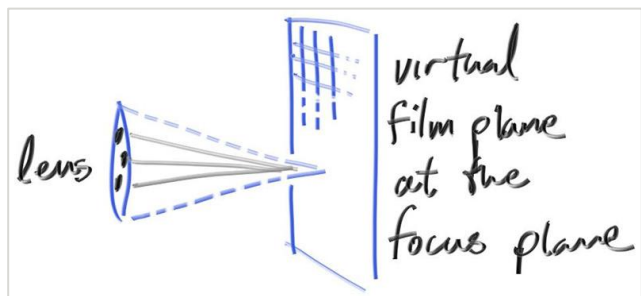
카메라 설정

Defocus Blur

초점 흐림 — 초점 평면과 거리



카메라 내부 · 외부 모습 — 구현할 때 내부는 시뮬레이션하지 않는다



초점 평면에 놓인 가상의 필름 평면

초점 평면

모든 것이 완벽하게 초점이 맞는 평면

초점 거리
`focus_dist`

카메라 중심과 초점 평면 사이의 거리

초점 길이
`focal_length`

카메라 중심과 이미지 평면 사이의 거리

$\text{focus_dist} \neq \text{focal_length}$

두 값은 원래 같지 않지만, 이 책에서는 픽셀 그리드를 초점 평면에 바로 배치할 것이라서 두 값이 같다.

Defocus Blur

초점 흐림 — Defocus 디스크와 원뿔 각도

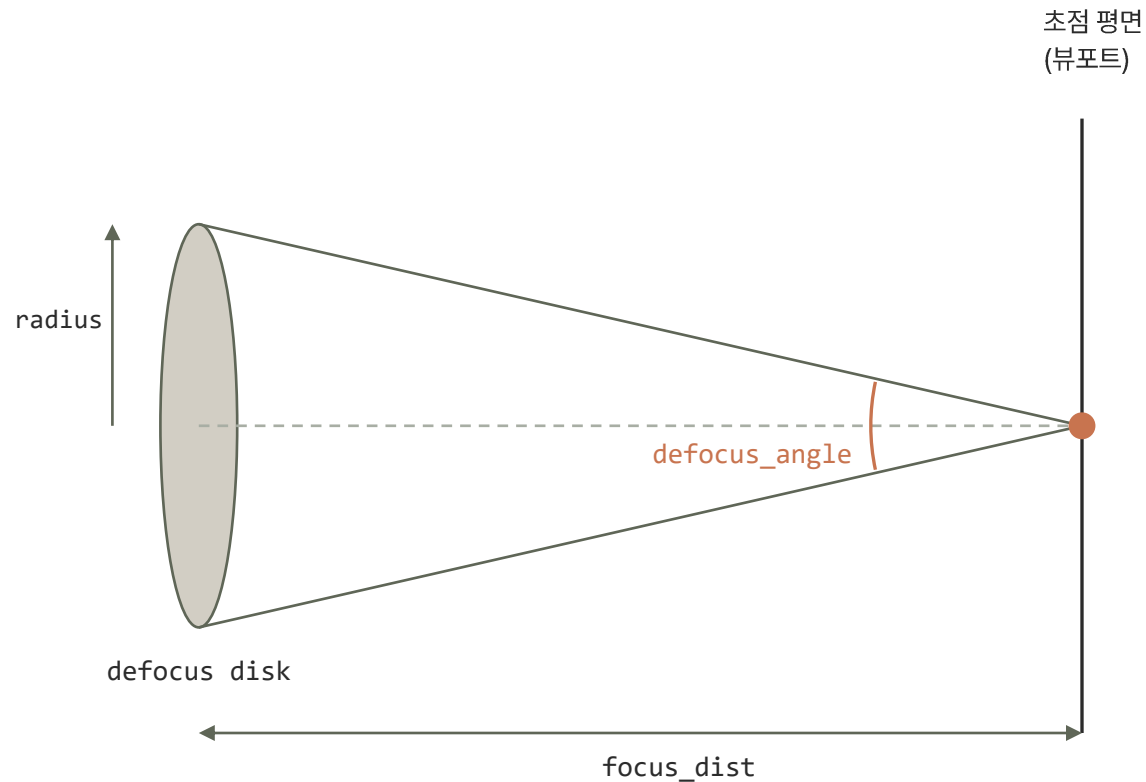
13

1. Defocus 디스크

Defocus Blur를 구현하기 위해서 카메라 중심을 중심으로 하는 디스크를 만든다. 디스크의 반지름이 클수록 Defocus Blur가 더 커진다.

2. 원뿔 각도로 지정

디스크의 반지름을 매개변수로 가질 수도 있지만, 뷰포트 중심을 꼭짓점으로 하고 Defocus 디스크를 밑면으로 하는 원뿔의 각도를 정한다. 이렇게 하면 특정 샷에 대해 초점 거리를 변경할 때 더 일관된 결과를 얻을 수 있다.



```
const double defocusRadius =  
    focus_dist * std::tan(DegreesToRadians(defocus_angle * 0.5));  
  
mDefocusDiskU = u * defocusRadius;  
mDefocusDiskV = v * defocusRadius;
```

defocusRadius

0.29746145598814161

const double

디버거로 확인한 defocusRadius 값

$$\text{radius} = \text{focus_dist} \cdot \tan(\text{defocus_angle} / 2)$$

Defocus Blur

초점 흐림 — 구현

13

1. 디스크 위 무작위 점

디스크에서 무작위 점을 고르는 데 원에서 고르는 것이므로, 무작위 2차원 좌표를 생성하는 것과 같다.

```
inline Vector3 RandomInUnitDisk()
{
    while (true)
    {
        auto p = Vector3(RandomDouble(-1, 1), RandomDouble(-1, 1), 0);
        if (p.LengthSquared() < 1)
        {
            return p;
        }
    }
}
```

2. 광선의 시작점

defocus_angle이 0 이하면 카메라 중심에서, 아니면 Defocus 디스크 위의 무작위 점에서 광선을 쏜다.

```
auto rayOrigin = (defocus_angle <= 0.0) ? mCenter : DefocusDiskSample();
auto rayDirection = pixelSample - rayOrigin;
```

3. 이전 구현과의 관계

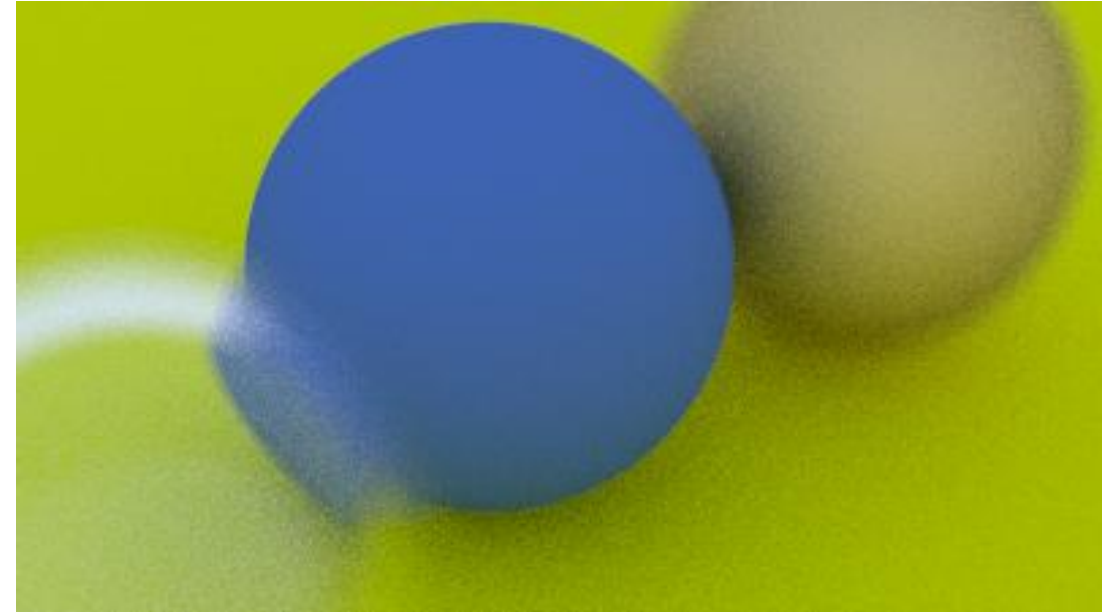
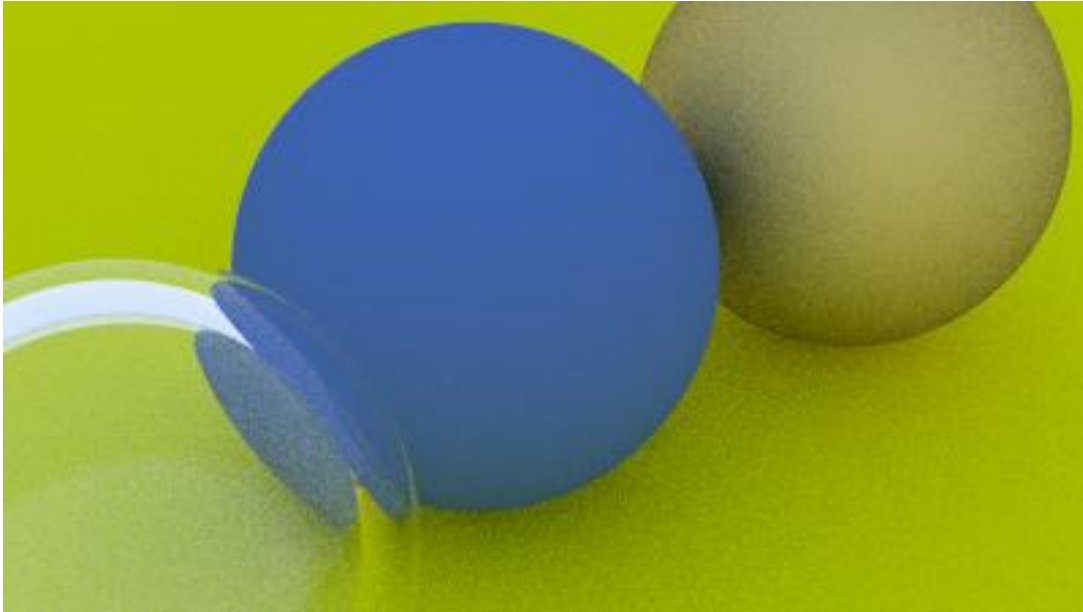
이전 구현은 디스크의 반지름 길이가 0인 Defocus 디스크였던 것을 알 수 있다.

```
Point3 DefocusDiskSample() const
{
    const Vec3 point = RandomInUnitDisk();
    return mCenter + (point.X() * mDefocusDiskU) + (point.Y() * mDefocusDiskV);
}
```

Defocus Blur

초점 흐림 — 결과

13



Before

초점 흐림 없음 — 모든 물체가 선명하다.

After

Defocus Blur 적용 — 초점 평면에서 벗어난 물체가 흐려진다.

Final Render

최종 렌더 — 무작위 구 장면

14

1. 장면 구성

22 × 22 격자에 작은 구를 무작위로 배치한다(확산 80% · 금속 15% · 유리 5%). 가운데에 큰 구 3개(유리 · 확산 · 금속)를 둔다.

2. 해상도와 샘플

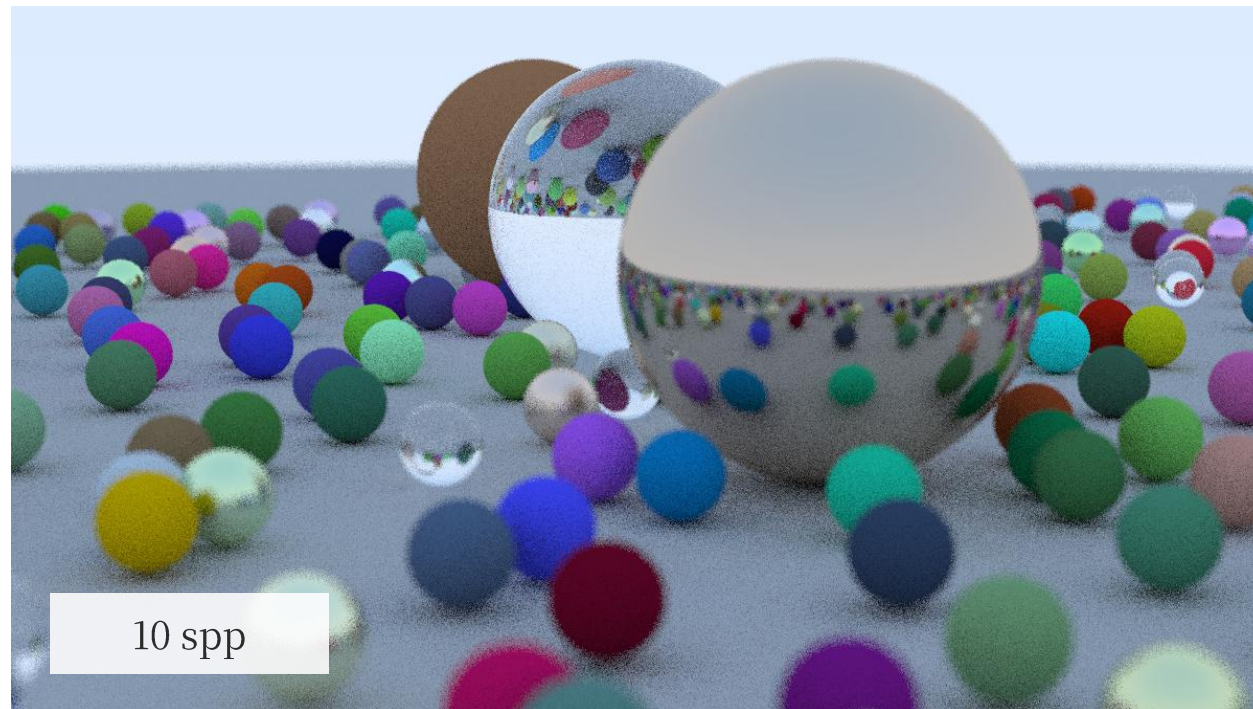
1200 × 675, 최대 반사 깊이 50. 픽셀당 샘플링을 10회에서 500회로 늘려 노이즈를 없앴다.

3. 카메라

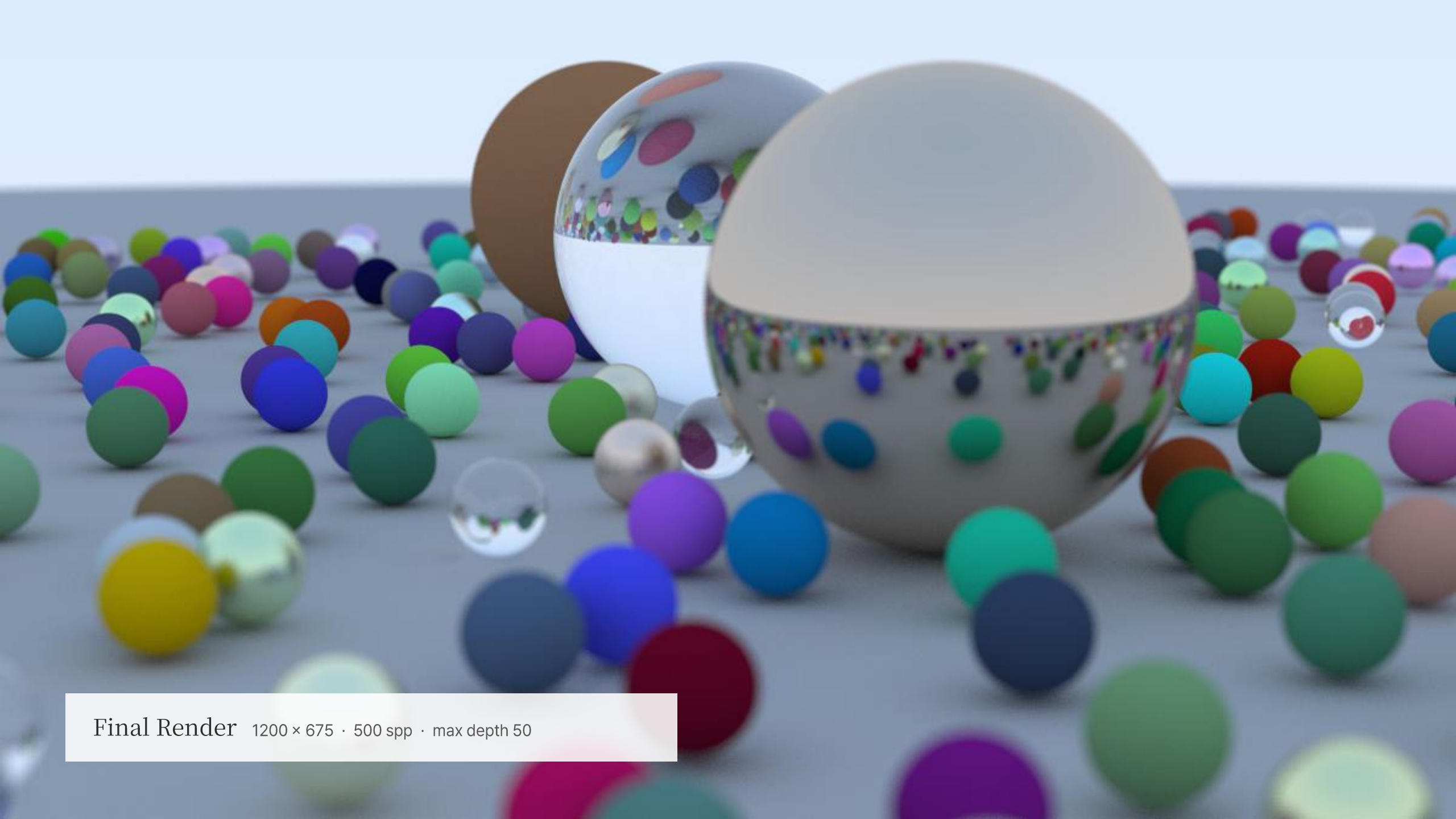
vFov 20°, lookFrom (13, 2, 3) → lookAt (0, 0, 0)

4. 초점 흐림

defocus_angle 0.6, focus_dist 10



픽셀당 샘플링 10회 — 노이즈가 남아 있다. 500회 결과는 다음 장.



Final Render 1200 × 675 · 500 spp · max depth 50